

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گروه مهندسی نرم افزار

پایان نامه کارشناسی

رشته مهندسی کامپیوتر گرایش نرم افزار

عنوان پایان نامه

سامانه‌ی پرسش محور مبتنی بر RAG برای توضیح کدهای پایتون

استاد راهنما:

دکتر محمدرضا شعرباف

پژوهشگر:

فاطمه صیادزاده

شهریور ۱۴۰۵



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گروه مهندسی نرم افزار

پروژه کارشناسی رشته‌ی مهندسی کامپیوتر گرایش نرم افزار

خانم فاطمه صیادزاده

تحت عنوان

سامانه‌ی پرسش محور مبتنی بر RAG برای توضیح کدهای پایتون

در تاریخ / / ۱۴ توسط هیأت داوران زیر بررسی و با نمره به تصویب نهایی رسید.

۱- استاد راهنمای پروژه:

دکتر

امضا

۲- استاد داور :

دکتر

امضا

امضای مدیر گروه

تشکر و قدردانی

بدین وسیله مراتب سپاس و قدردانی خود را از استاد راهنمای گرامی، جناب آقای دکتر محمدرضا شهرباف، به پاس راهنمایی‌های علمی، نکات ارزشمند و حمایت‌های ایشان در مراحل مختلف انجام این پروژه اعلام می‌دارم. همچنین از استاد داور محترم، به سبب زمانی که برای بررسی این گزارش اختصاص داده‌اند و نکات و پیشنهادهای ارزشمندشان، صمیمانه تشکر و قدردانی می‌کنم.

از خانواده عزیزم نیز که با صبوری، دلگرمی و حمایت همیشگی خود زمینه پیمودن این مسیر را فراهم کردند، سپاسگزارم. در پایان، از دوستان و همکارانی که در طول انجام پروژه با ارائه مشورت، طرح دیدگاه‌های سازنده و همراهی خود به پیشبرد این کار کمک کردند، صمیمانه قدردانی می‌کنم.

تقدیم به خانواده عزیزم؛

که با عشق، صبوری و حمایت همیشگی خود،

در تمام مراحل این مسیر همراه و پشتیبان من بودند.

چکیده:

درک یک مخزن نرم‌افزاری واقعی تنها با خواندن چند فایل یا جست‌وجوی واژه‌های یکسان امکان‌پذیر نیست؛ زیرا رفتار سامانه میان ماژول‌ها، کلاس‌ها و توابع توزیع شده است و پرسش طبیعی کاربر، به‌ویژه به زبان فارسی، ممکن است با نام‌های انگلیسی موجود در کد هم‌پوشانی واژگانی نداشته باشد. از سوی دیگر، ارسال مستقیم کل مخزن به مدل زبانی با محدودیت ظرفیت ورودی، هزینه و احتمال تولید ادعای بدون پشتوانه روبه‌رو است. در این پروژه، سامانه CodeCompass به‌عنوان یک راهکار محلی‌محور برای درک مخازن پایتون طراحی و پیاده‌سازی شد. سامانه پس از پویش امن مخزن، ساختار کد را با درخت نحو انتزاعی استخراج و قطعه‌های ساختارآگاه و دارای شناسه پایدار تولید می‌کند. فراداده فایل‌ها، نمادها، هش محتوا و محدوده خطوط در SQLite به‌عنوان منبع اصلی حقیقت نگهداری می‌شود و ChromaDB تنها نقش نمایه برداری قابل جایگزینی را دارد. بازیابی شواهد به سه روش واژگانی، معنایی و ترکیبی انجام می‌شود و مدل زبانی با استفاده از زمینه منتخب، پاسخ فارسی یا انگلیسی و مستندات تابع را تولید می‌کند. مسیر فایل، نام نماد و شماره خطوط از خروجی مدل استخراج نمی‌شوند و به‌صورت قطعی از فراداده سامانه ساخته می‌شوند؛ بنابراین کاربر می‌تواند هر ارجاع را مستقیماً در نمایشگر کد بررسی کند. ارزیابی نهایی نشان داد تعداد بردارها در هر نه نمایه آزمایش‌شده با تعداد قطعه‌های قطعی کاملاً منطبق است. مدل Gemini Embedding2 در بازیابی معنایی نرخ موفقیت رتبه سوم ۹۴/۴ درصد و میانگین معکوس رتبه ۰/۸۵۳ را ثبت کرد و ۷۱ پاسخ از ۷۲ اجرای پرسش‌وپاسخ قابل استفاده بود. در ارزیابی انسانی بر مقیاس صفر تا ده، میانگین امتیاز ترکیبی کیفیت پاسخ برای بازیابی معنایی ۸/۷۷ و برای بازیابی واژگانی ۴/۵۲ بود و این تفاوت از نظر آماری معنادار بود. ارسال کل مخزن در پاسخ‌های موفق مخازن کوچک کیفیت خام بیشتری ایجاد کرد، اما برای مخزن بزرگ CodeCompass قابل اجرا نبود؛ در مقابل، RAG معنایی با نزدیک به ۹۵ درصد ورودی کمتر، هزینه و زمان پایین‌تر و قابلیت استفاده در مخزن بزرگ، راهکار عملی‌تری بود. عامل ابزارمحور نیز برتری کیفی معناداری نسبت به RAG معنایی نشان نداد و کندتر و پرهزینه‌تر بود. در مجموع، نتایج از انتخاب بازیابی معنایی مبتنی بر شواهد به‌عنوان مسیر پیش‌فرض CodeCompass و از جداسازی متن احتمالی مدل از ارجاع‌های قطعی منبع پشتیبانی می‌کنند.

واژگان کلیدی: درک کد، تولید تقویت‌شده با بازیابی، بازیابی معنایی، تحلیل ساختاری کد، مدل

زبانی بزرگ، ارجاع قطعی

فهرست مطالب

صفحه	عنوان
۱	فصل اول مقدمه.....
۱	۱-۱- بیان مسئله.....
۲	۲-۱- هدف پروژه.....
۳	۳-۱- کاربردهای پروژه.....
۳	۴-۱- ساختار پایان نامه.....
۴	فصل دوم مفاهیم.....
۴	۱-۲- مقدمه.....
۵	۲-۲- درک و بازیابی کد.....
۵	۱-۲-۲- درک پایگاه کد.....
۵	۲-۲-۲- جست و جوی کد.....
۶	۳-۲- تحلیل ساختاری کد با درخت نحو انتزاعی.....
۷	۴-۲- قطع بندی ساختار آگاه کد.....
۷	۵-۲- نمایش برداری و تعبیه سازی.....
۸	۶-۲- معیارهای شباهت برداری.....
۸	۱-۶-۲- شباهت کسینوسی.....
۹	۲-۶-۲- فاصله اقلیدسی.....
۹	۳-۶-۲- ضرب داخلی.....
۹	۴-۶-۲- معیارهای دیگر.....
۱۰	۷-۲- روش های بازیابی اطلاعات.....
۱۰	۱-۷-۲- بازیابی واژگانی.....
۱۰	۲-۷-۲- بازیابی معنایی.....
۱۱	۳-۷-۲- بازیابی ترکیبی.....
۱۲	۸-۲- تولید تقویت شده با بازیابی.....
۱۳	۹-۲- مدل های زبانی و تولید توضیح کد.....
۱۳	۱۰-۲- پرسش های فارسی درباره کد انگلیسی.....
۱۴	۱۱-۲- پژوهش های مرتبط.....

فهرست مطالب

صفحه	عنوان
۱۴	CodeSearchNet - ۱-۱۱-۲
۱۴	CodeBERT - ۲-۱۱-۲
۱۴	GraphCodeBERT - ۳-۱۱-۲
۱۴	ProConSuL - ۴-۱۱-۲
۱۵	۵-۱۱-۲- تولید تقویت‌شده با بازیابی
۱۵	۱۲-۲- جایگاه CodeCompass و شکاف پژوهشی
۱۶	۱۳-۲- جمع‌بندی
۱۷	فصل سوم طراحی و پیاده‌سازی سامانه CodeCompass
۱۷	۱-۳- مقدمه
۱۷	۲-۳- نیازمندی‌های سامانه
۱۸	۱-۲-۳- نیازمندی‌های کارکردی
۱۸	۲-۲-۳- نیازمندی‌های غیرکارکردی
۱۹	۳-۲-۳- ملاحظات کارایی و مقیاس‌پذیری
۱۹	۳-۳- محدوده سامانه و موارد خارج از محدوده
۲۰	۴-۳- معماری کلی CodeCompass
۲۱	۱-۴-۳- سهم فنی و تصمیم‌های طراحی سامانه
۲۱	۵-۳- جریان کامل عملکرد سامانه
۲۱	۱-۵-۳- انتخاب مخزن
۲۱	۲-۵-۳- تحلیل و نمایه‌سازی
۲۱	۳-۵-۳- بازیابی شواهد
۲۲	۴-۵-۳- تولید پاسخ
۲۲	۵-۵-۳- ساخت ارجاع
۲۲	۶-۵-۳- نمایش کد منبع
۲۲	۶-۳- پویای امن مخزن
۲۲	۱-۶-۳- کنترل مسیر ریشه
۲۲	۲-۶-۳- فایل‌های قابل پذیرش

فهرست مطالب

عنوان	صفحه
۳-۶-۳- حذف فایل‌های محرمانه، محیط مجازی، حافظه‌های نهان و خروجی ساخت.....	۲۳
۷-۳- تحلیل ساختاری کد پایتون.....	۲۳
۱-۷-۳- استخراج واحدهای مازولی.....	۲۳
۲-۷-۳- کلاس‌ها.....	۲۳
۳-۷-۳- توابع و متدها.....	۲۳
۴-۷-۳- امضا، پارامترها، رشته‌ی مستندات و محدوده خطوط.....	۲۳
۸-۳- قطعه‌بندی ساختار آگاه.....	۲۴
۱-۸-۳- تولید شناسه پایدار.....	۲۴
۲-۸-۳- هش محتوا.....	۲۴
۳-۸-۳- ارتباط قطعه با فایل و نماد.....	۲۴
۹-۳- طراحی لایه ذخیره‌سازی.....	۲۴
۱-۹-۳- SQLite به عنوان منبع اصلی حقیقت.....	۲۴
۲-۹-۳- ChromaDB به عنوان نمایه برداری قابل جایگزینی.....	۲۵
۱۰-۳- فرایند نمایه‌سازی.....	۲۵
۱-۱۰-۳- نمایه‌سازی اولیه.....	۲۶
۲-۱۰-۳- نمایه‌سازی افزایشی.....	۲۶
۳-۱۰-۳- ساخت نمایه‌ی نامزد.....	۲۶
۴-۱۰-۳- اعتبارسنجی پیش از فعال‌سازی.....	۲۷
۵-۱۰-۳- حفظ نمایه قبلی در صورت شکست.....	۲۷
۱۱-۳- روش‌های بازیابی.....	۲۷
۱-۱۱-۳- بازیابی واژگانی.....	۲۷
۲-۱۱-۳- بازیابی معنایی.....	۲۷
۳-۱۱-۳- بازیابی ترکیبی.....	۲۸
۴-۱۱-۳- محدودسازی و رتبه‌بندی نتایج.....	۲۸
۱۲-۳- ساخت زمینه برای RAG.....	۲۸
۱۳-۳- تولید پاسخ مستند.....	۲۸

فهرست مطالب

عنوان	صفحه
۳-۱۳-۱- پاسخ مبتنی بر شواهد.....	۲۹
۳-۱۳-۲- رفتار شواهد ناکافی.....	۲۹
۳-۱۳-۳- جداسازی متن مدل از اطلاعات قطعی.....	۲۹
۳-۱۴-۱- تولید ارجاع قابل اعتماد.....	۳۰
۳-۱۴-۱- مسیر فایل.....	۳۰
۳-۱۴-۲- نام نماد.....	۳۰
۳-۱۴-۳- شماره خطوط.....	۳۰
۳-۱۴-۴- جلوگیری از جعل ارجاع توسط مدل.....	۳۰
۳-۱۵-۱- تولید مستندات تابع.....	۳۰
۳-۱۵-۱- استخراج قطعی اطلاعات تابع.....	۳۰
۳-۱۵-۲- تولید توضیح توسط مدل.....	۳۱
۳-۱۵-۳- اعتبارسنجی ساختار خروجی.....	۳۱
۳-۱۶-۱- معماری ارائه‌دهنده‌ها.....	۳۱
۳-۱۶-۱- اولاما (Ollama).....	۳۱
۳-۱۶-۲- ارائه‌دهندگان سازگار با OpenAI.....	۳۱
۳-۱۶-۳- استقلال مدل تعبیه‌سازی و LLM.....	۳۲
۳-۱۷-۱- رابط برنامه‌نویسی FastAPI.....	۳۲
۳-۱۸-۱- رابط کاربری React و Vite.....	۳۳
۳-۱۸-۱- مدیریت پروژه.....	۳۳
۳-۱۸-۲- مرور فایل‌ها و نمادها.....	۳۳
۳-۱۸-۳- جست‌وجو (Search).....	۳۴
۳-۱۸-۴- پرسش‌وپاسخ (Ask).....	۳۴
۳-۱۸-۵- مستندسازی (Documentation).....	۳۴
۳-۱۸-۶- نمایش خطوط مورد استناد با Monaco.....	۳۴
۳-۱۸-۷- داشبورد ارزیابی.....	۳۴
۳-۱۹-۱- ملاحظات امنیت، حریم خصوصی و مدیریت خطا.....	۳۴

فهرست مطالب

عنوان	صفحه
۳-۲۰- جمع‌بندی.....	۳۵
فصل چهارم ارزیابی و نتایج پیاده‌سازی سامانه CodeCompass.....	۳۶
۴-۱- مقدمه.....	۳۶
۴-۲- اهداف و پرسش‌های ارزیابی.....	۳۶
۴-۳- طراحی ارزیابی.....	۳۷
۴-۳-۱- مخازن مورد ارزیابی.....	۳۷
۴-۳-۲- مجموعه آزمون و اندازه آزمایش.....	۳۷
۴-۳-۳- مدل‌ها و تنظیمات.....	۳۸
۴-۳-۴- معیارهای ارزیابی.....	۳۸
۴-۴- نتایج عملکرد کارکردی سامانه.....	۳۸
۴-۵- نتایج نمایه‌سازی و قابلیت اعتماد داده‌ها.....	۳۹
۴-۶- نتایج ارزیابی بازیابی.....	۴۰
۴-۶-۱- نتایج کلی.....	۴۰
۴-۶-۲- نتایج دوزبانه.....	۴۱
۴-۶-۳- زمان بازیابی.....	۴۱
۴-۶-۴- مجموعه ارزیابی رسمی بازیابی با ۶۰ پرسش.....	۴۲
۴-۷- نتایج پرسش‌وپاسخ مستند.....	۴۲
۴-۷-۱- قابلیت اجرای مدل‌ها.....	۴۲
۴-۷-۲- کیفیت پاسخ بر اساس مدل زبانی.....	۴۳
۴-۷-۳- ارتباط کیفیت بازیابی و پاسخ.....	۴۳
۴-۷-۴- تحلیل خطا و توهم.....	۴۳
۴-۸- نتایج مستندسازی تابع.....	۴۴
۴-۹- آزمایش مقایسه راهبردهای ساخت زمینه.....	۴۴
۴-۹-۱- روش آزمایش.....	۴۴
۴-۹-۲- اعتبارسنجی انسانی کور مستقل.....	۴۴
۴-۹-۳- کل مخزن در برابر RAG معنایی.....	۴۵

فهرست مطالب

عنوان	صفحه
۴-۹-۴ سودمندی تعبیه‌سازی.....	۴۵
۵-۹-۴ عامل OpenCode در برابر RAG معنایی.....	۴۶
۶-۹-۴ هزینه و زمان راهبردها.....	۴۶
۱۰-۴ بحث و تفسیر نتایج.....	۴۷
۱۱-۴ محدودیت‌ها و تهدیدهای اعتبار.....	۴۷
۱۲-۴ جمع‌بندی.....	۴۸
فصل پنجم نتیجه‌گیری و پیشنهادها.....	۴۹
۱-۵ نتیجه‌گیری کلی.....	۴۹
۲-۵ میزان تحقق اهداف پروژه.....	۴۹
۳-۵ پاسخ به پرسش‌های اصلی ارزیابی.....	۵۰
۱-۳-۵ انتخاب زمینه با RAG در برابر ارسال کل مخزن.....	۵۰
۲-۳-۵ سودمندی تعبیه‌سازی در مقایسه با بازیابی واژگانی.....	۵۱
۳-۳-۵ عامل ابزارمحور در برابر RAG معنایی.....	۵۱
۴-۵ نوآوری و سهم پروژه.....	۵۱
۵-۵ محدودیت‌ها.....	۵۲
۶-۵ پیشنهادها برای ادامه کار.....	۵۲
منابع.....	۵۴

فهرست شکل‌ها

عنوان	صفحه
شکل ۱-۲: نمونه تبدیل کد پایتون به ساختار AST.....	۶
شکل ۲-۲: مفهوم زاویه میان بردار پرسش و قطعه کد در شباهت کسینوسی.....	۱۰
شکل ۳-۲: فرایند عمومی پاسخ‌گویی مبتنی بر RAG در یک سامانه درک کد.....	۱۲
شکل ۱-۳: معماری کلی سامانه CodeCompass.....	۲۰
شکل ۲-۳: جریان نمایه‌سازی امن و فعال‌سازی اتمی.....	۲۶
شکل ۳-۳: جریان پرسش تا پاسخ مستند و نمایش منبع.....	۲۹
شکل ۴-۳: تنظیم مستقل ارائه‌دهنده تعبیه‌سازی و مدل زبانی.....	۳۲
شکل ۱-۴: فضای کاری اصلی و انتخاب روش بازیابی در CodeCompass.....	۳۹
شکل ۲-۴: داشبورد نتایج نهایی ارزیابی پایان‌نامه.....	۳۹
شکل ۳-۴: داشبورد نتایج ارزیابی رسمی بازیابی.....	۴۲
شکل ۴-۴: داشبورد مقایسه راهبردهای ساخت زمینه.....	۴۷

فهرست جدول‌ها

عنوان	صفحه
جدول ۱-۲: مقایسه روش‌های بازیابی مورد استفاده در CodeCompass	۱۱
جدول ۲-۲: مقایسه CodeCompass با پژوهش‌های مرتبط	۱۵
جدول ۱-۳: نیازمندی‌های کارکردی CodeCompass	۱۸
جدول ۲-۳: نیازمندی‌های غیرکارکردی CodeCompass	۱۹
جدول ۳-۳: تقسیم مسئولیت SQLite و ChromaDB	۲۵
جدول ۴-۳: گروه‌های اصلی API سامانه	۳۲
جدول ۱-۴: مشخصات مخازن ارزیابی نهایی	۳۷
جدول ۲-۴: اندازه مجموعه ارزیابی نهایی	۳۸
جدول ۳-۴: نتیجه اعتبارسنجی نمایه‌ها	۴۰
جدول ۴-۴: نتایج کلی بازیابی روی ۳۶ پرسش	۴۰
جدول ۵-۴: عملکرد بازیابی بر اساس زبان پرسش	۴۱
جدول ۶-۴: قابلیت اجرای مدل‌ها در پرسش‌وپاسخ	۴۳
جدول ۷-۴: میانگین ارزیابی انسانی کیفیت پاسخ‌ها	۴۳
جدول ۸-۴: نتایج اعتبارسنجی انسانی راهبردهای زمینه	۴۵
جدول ۹-۴: هزینه، زمان و قابلیت اجرای راهبردها	۴۶

مخفف‌ها:

مخفف

عبارت کامل انگلیسی

API	Application Programming Interface
AST	Abstract Syntax Tree
CORS	Cross-Origin Resource Sharing
GLM	General Language Model
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
MRR	Mean Reciprocal Rank
POSIX	Portable Operating System Interface
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SHA-256	Secure Hash Algorithm 256-bit
URL	Uniform Resource Locator
VCS	Version Control System

فصل اول

مقدمه

درک یک پایگاه کد نرم‌افزاری، به‌ویژه هنگامی که پروژه بزرگ شده یا مستندات کافی ندارد، فرایندی زمان‌بر است. رفتار یک سامانه ممکن است میان چندین فایل، کلاس و تابع توزیع شده باشد و توسعه‌دهنده برای یافتن محل پیاده‌سازی یک قابلیت یا بررسی علت یک رفتار، ناچار شود بخش‌های مختلف مخزن را مطالعه کند. این موضوع در نگهداشت نرم‌افزار، رفع خطا، توسعه قابلیت‌های جدید و آشنایی اعضای تازه با پروژه اهمیت دارد.

پروژه CodeCompass برای کمک به درک مخازن کد پایتون طراحی شده است. این سامانه میان پرسش طبیعی کاربر و شواهد واقعی موجود در کد ارتباط برقرار می‌کند و پاسخی همراه با مسیر فایل، نام نماد¹ و محدوده خطوط ارائه می‌دهد. در این فصل، مسئله، هدف، دستاوردها، کاربردها و ساختار گزارش معرفی می‌شوند.

۱-۱- بیان مسئله

ابزارهای جست‌وجوی متنی زمانی مناسب‌اند که کاربر نام دقیق فایل، تابع یا شناسه موردنظر را بدانند؛ اما نیاز توسعه‌دهنده معمولاً به‌صورت یک پرسش رفتاری بیان می‌شود. برای مثال، ممکن است کاربر بپرسد «توکن دسترسی پس از ورود کاربر در کجا ساخته می‌شود؟»، درحالی‌که این رفتار در کد با نام‌هایی مانند `create_access_token` یا `issue_jwt` پیاده‌سازی شده باشد. در این وضعیت، تطبیق مستقیم کلمات پرسش با متن کد کافی نیست. این فاصله زمانی بیشتر می‌شود که پرسش فارسی و شناسه‌های کد انگلیسی باشند.

¹ Symbol

از سوی دیگر، مدل‌های زبانی^۱ می‌توانند کد را به زبان طبیعی توضیح دهند، اما در صورت استفاده بدون شواهد کافی ممکن است پاسخی ناسازگار با پیاده‌سازی واقعی مخزن تولید کنند. همچنین، واگذاری اطلاعاتی مانند مسیر فایل، نام نماد و محدوده خطوط به مدل می‌تواند به تولید مشخصات نادرست منجر شود. بنابراین، پاسخ باید به شواهد واقعی مخزن متصل باشد و اطلاعات منبع از داده‌های قطعی استخراج شوند، نه از متن تولیدشده توسط مدل.

مسئله اصلی این پروژه، طراحی سامانه‌ای است که بتواند یک مخزن محلی پایتون را به صورت ساختاری تحلیل کند، پرسش فارسی یا انگلیسی کاربر را به قطعه‌های مرتبط کد متصل سازد و پاسخی مستند و قابل بررسی ارائه دهد. در این سامانه، مدل زبانی وظیفه نگارش توضیح را بر عهده دارد، اما مسیر فایل، نام نماد، محدوده خطوط و ارجاع‌ها از فراداده استخراج شده از مخزن تولید می‌شوند.

همچنین، یک روش بازیابی برای همه پرسش‌ها مناسب نیست. بازیابی واژگانی در یافتن نام‌های دقیق مؤثر است، اما ممکن است ارتباط مفهومی پرسش و کد را تشخیص ندهد. بازیابی معنایی این ارتباط را بهتر شناسایی می‌کند، ولی ممکن است در تطبیق شناسه‌های دقیق خطا کند. بنابراین، ترکیب و مقایسه این روش‌ها نیز بخشی از مسئله پروژه است.

تمرکز CodeCompass بر پرسش‌های محلی و نمادمحور، مانند یافتن پیاده‌سازی یک رفتار یا توضیح تابع و کلاس است. سامانه پوشش تضمین‌شده‌ای برای تحلیل کامل معماری کلان مخزن یا استخراج عمومی طرح پایگاه داده ندارد. اگر شواهد بازیابی شده کافی نباشند، باید این محدودیت را صریح اعلام کند و از تولید پاسخ حدسی خودداری نماید.

۱-۲- هدف پروژه

هدف اصلی پروژه، طراحی و پیاده‌سازی یک سامانه محلی و پرسش‌محور مبتنی بر تولید تقویت‌شده با بازیابی^۲ برای درک کدهای پایتون است. کاربر باید بتواند درباره منطق یک مخزن به زبان فارسی یا انگلیسی پرسش کند و در پاسخ، توضیحی مبتنی بر شواهد همراه با محل دقیق منبع دریافت نماید.

برای دستیابی به این هدف، فایل‌های پایتون در محدوده مخزن انتخاب شده پیمایش می‌شوند و با استفاده از درخت نحو انتزاعی^۳، اطلاعات کلاس‌ها، توابع، متدها و محدوده خطوط استخراج می‌گردند. سپس قطعه‌های ساختارآگاه ایجاد شده و فراداده معتبر آن‌ها در SQLite نگهداری می‌شود. برای یافتن شواهد نیز روش‌های واژگانی، معنایی و ترکیبی در نظر گرفته شده‌اند.

در مرحله تولید پاسخ، تنها شواهد دارای رتبه بالاتر در اختیار مدل زبانی قرار می‌گیرند. مدل توضیح

¹ Language Models

² Retrieval-Augmented Generation (RAG)

³ Abstract Syntax Tree (AST)

طبیعی را تولید می‌کند، درحالی‌که مشخصات منبع و ارجاع‌ها از فراداده قطعی سامانه ساخته می‌شوند. سازگاری مدل تعبیه‌ساز^۱ با نمایه برداری نیز پیش از بازیابی بررسی می‌شود.

دستآورد پروژه، پیاده‌سازی یک سامانه یکپارچه برای تحلیل، بازیابی و توضیح مخازن پایتون است. این سامانه نمایه‌سازی افزایشی، بازیابی واژگانی، معنایی و ترکیبی، پاسخ‌گویی فارسی و انگلیسی، ارجاع قطعی، نمایش محل شواهد و تولید مستندات اولیه توابع را فراهم می‌کند. همچنین، روش‌های بازیابی و مدل‌های تولید پاسخ در یک فرایند تکرارپذیر ارزیابی شده‌اند. CodeCompass مدل جدیدی آموزش نمی‌دهد و الگوریتم تازه‌ای ارائه نمی‌کند؛ دستآورد آن، ترکیب کنترل‌شده اجزای موجود در سامانه‌ای قابل‌استفاده و قابل‌بررسی است.

۱-۳- کاربردهای پروژه

یکی از کاربردهای اصلی CodeCompass یافتن محل پیاده‌سازی یک رفتار در مخزن است. کاربر می‌تواند بدون دانستن نام دقیق تابع، پرسش خود را با زبان طبیعی مطرح کند و قطعه‌های مرتبط را همراه با مسیر فایل و محدوده خطوط مشاهده نماید.

سامانه در نگهداشت نرم‌افزار و آشنایی اعضای جدید با پروژه نیز کاربرد دارد. توسعه‌دهنده می‌تواند درباره نقش یک تابع یا کلاس سؤال کند و پاسخ را از طریق ارجاع به منبع بررسی نماید. همچنین، امکان تولید مستندات اولیه برای توابع بر اساس اطلاعاتی مانند امضا، پارامترها، نوع بازگشتی و خطاهای صریح وجود دارد. CodeCompass برای آموزش و مطالعه پروژه‌های پایتون نیز قابل استفاده است و امکان طرح پرسش فارسی درباره کد انگلیسی را فراهم می‌کند. بااین‌حال، پاسخ سامانه جایگزین بازبینی مستقیم کد نیست و در تصمیم‌های فنی مهم باید شواهد ارجاع‌شده بررسی شوند.

۱-۴- ساختار پایان‌نامه

این گزارش در پنج فصل تنظیم شده است. فصل اول مسئله، هدف، دستاوردها و کاربردهای پروژه را معرفی می‌کند. فصل دوم به مفاهیم نظری و پژوهش‌های مرتبط اختصاص دارد. در فصل سوم، معماری و نحوه پیاده‌سازی CodeCompass تشریح می‌شود. فصل چهارم روش ارزیابی، معیارها و نتایج پروژه را ارائه می‌کند و فصل پنجم شامل جمع‌بندی، پاسخ به پرسش‌های پژوهش، محدودیت‌ها و پیشنهادهای ادامه کار است.

¹ Embedding Model

فصل دوم

مفاهیم

۲-۱- مقدمه

درک یک پایگاه کد نرم‌افزاری صرفاً به معنای خواندن دستورات موجود در فایل‌های آن نیست. در یک پروژه واقعی، رفتار سامانه ممکن است میان چندین فایل، کلاس، تابع و لایه نرم‌افزاری توزیع شده باشد. افزون بر این، نام‌هایی که توسعه‌دهندگان در کد استفاده می‌کنند، همواره با عباراتی که کاربران برای بیان پرسش خود به کار می‌برند یکسان نیستند. برای مثال، کاربر ممکن است درباره محل «ساخت توکن کاربر» سؤال کند، در حالی که این رفتار در کد با شناسه‌هایی مانند `create_access_token` یا `issue_jwt` پیاده‌سازی شده باشد. در چنین شرایطی، جست‌وجوی صرفاً متنی نمی‌تواند همواره ارتباط میان مفهوم موردنظر کاربر و شناسه‌های موجود در کد را تشخیص دهد.

جست‌وجوی معنایی کد با هدف کاهش این فاصله شکل گرفته است. در این رویکرد، سیستم تلاش می‌کند ارتباط میان یک پرسش زبان طبیعی و قطعه‌های کد را بر اساس معنا تشخیص دهد، نه اینکه تنها به وجود کلمات یکسان در آن‌ها متکی باشد. پروژه `CodeSearchNet` نیز جست‌وجوی معنایی کد را مسئله بازیابی قطعه کد مرتبط بر اساس یک پرسش زبان طبیعی معرفی کرده و بر دشواری تطبیق زبان فنی کد با بیان طبیعی کاربر تأکید کرده است [1].

از سوی دیگر، مدل‌های زبانی بزرگ توانایی مناسبی در تولید توضیح طبیعی درباره کد دارند؛ اما اگر بدون شواهد کافی مورد استفاده قرار گیرند، ممکن است توضیحاتی تولید کنند که با محتوای واقعی مخزن سازگار نباشد. معماری تولید تقویت‌شده با بازیابی یا RAG با ترکیب یک مرحله بازیابی اطلاعات و یک مرحله تولید پاسخ، زمینه مرتبط را پیش از تولید پاسخ در اختیار مدل قرار می‌دهد [2].

در این فصل، مفاهیم نظری موردنیاز برای طراحی `CodeCompass` بررسی می‌شوند. ابتدا مسئله درک و

بازیابی کد مطرح شده و سپس تحلیل ساختاری کد، قطعه‌بندی¹ ساختار آگاه، نمایش برداری، معیارهای شباهت، روش‌های بازیابی و معماری RAG توضیح داده می‌شوند. در ادامه، نقش مدل‌های زبانی، چالش پرسش‌های فارسی و پژوهش‌های مرتبط بررسی شده و در پایان، جایگاه CodeCompass نسبت به کارهای پیشین مشخص می‌شود.

۲-۲- درک و بازیابی کد

۲-۲-۱- درک پایگاه کد

درک پایگاه کد فرایندی است که طی آن توسعه‌دهنده تلاش می‌کند ساختار، مسئولیت اجزا، روابط میان بخش‌ها و نحوه پیاده‌سازی رفتارهای نرم‌افزار را شناسایی کند. این فرایند در فعالیت‌هایی مانند نگهداشت نرم‌افزار، رفع خطا، توسعه قابلیت جدید، بازبینی کد و ورود اعضای جدید به تیم اهمیت دارد. هرچه پروژه بزرگ‌تر یا مستندات آن ناقص‌تر باشد، یافتن بخش مرتبط با یک رفتار خاص دشوارتر خواهد شد.

در پروژه‌های پایتون، واحدهای معنادار برنامه معمولاً شامل ماژول‌ها، کلاس‌ها، توابع و متدها هستند. برای پاسخ به یک پرسش درباره منطق پروژه، لازم است سیستم بتواند این واحدها را شناسایی کند و مشخصاتی مانند نام، موقعیت، پارامترها، محدوده خطوط و کد منبع آن‌ها را در اختیار فرایند بازیابی قرار دهد. در نتیجه، بازیابی کد نباید صرفاً در سطح فایل انجام شود؛ زیرا یک فایل ممکن است شامل چندین رفتار مستقل باشد و ارسال تمام آن به مدل، مقدار زیادی اطلاعات نامرتبط وارد زمینه پاسخ می‌کند.

۲-۲-۲- جست‌وجوی کد

هدف جست‌وجوی کد، یافتن بخش‌هایی از برنامه است که با نیاز اطلاعاتی کاربر ارتباط دارند. ورودی این فرایند می‌تواند نام یک تابع، بخشی از کد یا یک پرسش زبان طبیعی باشد. خروجی نیز معمولاً فهرستی رتبه‌بندی‌شده از فایل‌ها یا قطعه‌های کد است.

در جست‌وجوی سنتی، انطباق بر اساس حضور کلمات مشترک انجام می‌شود. این روش زمانی مؤثر است که کاربر نام دقیق فایل، تابع یا شناسه را بداند. با این حال، هنگامی که واژگان پرسش با واژگان کد متفاوت باشند، احتمال بازیابی‌نشدن نتیجه مرتبط افزایش می‌یابد. جست‌وجوی معنایی می‌کوشد این محدودیت را با نگاشت پرسش و کد به یک فضای برداری مشترک کاهش دهد.

CodeSearchNet یکی از کارهای پایه در این زمینه است. مجموعه ارائه‌شده در این پژوهش حدود شش میلیون تابع از شش زبان برنامه‌نویسی را شامل می‌شود و چالش آن با استفاده از پرسش‌های زبان طبیعی و قضاوت‌های انسانی درباره مرتبط‌بودن نتایج طراحی شده است [1]. اهمیت این پژوهش در آن است که

¹ Chunking

جست‌وجوی کد را از تطبیق ساده رشته‌ها فراتر برده و به‌عنوان مسئله‌ای میان زبان طبیعی و زبان برنامه‌نویسی مطرح می‌کند.

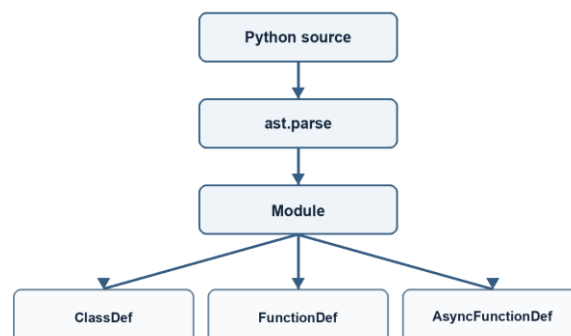
۲-۳- تحلیل ساختاری کد با درخت نحو انتزاعی

درخت نحو انتزاعی یا AST نمایش ساختاری یک برنامه پس از تحلیل نحوی است. در این نمایش، عناصر برنامه به‌صورت گره‌های یک درخت نشان داده می‌شوند. برای مثال، تعریف یک کلاس، تابع، شرط یا فراخوانی تابع با نوع مشخصی از گره نمایش داده می‌شود. برخلاف جست‌وجوی مستقیم در متن فایل، استفاده از AST امکان تشخیص نوع و مرز دقیق عناصر برنامه را فراهم می‌کند.

کتابخانه استاندارد پایتون ماژول ast را برای پردازش درخت نحو انتزاعی فراهم کرده است. تابع ast.parse متن برنامه را به مجموعه‌ای از گره‌های ساختاری تبدیل می‌کند و کلاس‌های این گره‌ها از ast.AST مشتق می‌شوند [3]. با پیمایش این درخت می‌توان اطلاعاتی مانند کلاس‌ها، توابع، توابع ناهمگام، متدها، پارامترها، نوع بازگشتی، تزئین‌گرها و دستورهای واردسازی را استخراج کرد.

استفاده از AST دو مزیت اصلی دارد. نخست، مرز واحدهای کد بر اساس ساختار واقعی زبان مشخص می‌شود و به قواعد شکننده متنی وابسته نیست. دوم، اطلاعات هویتی مانند نام کامل نماد و محدوده خطوط را می‌توان به‌صورت قطعی استخراج کرد. این اطلاعات بعداً برای ایجاد قطعه‌های کد و ارجاع کاربر به محل دقیق منبع استفاده می‌شوند.

باین‌حال، AST به‌تنهایی تمام روابط معنایی یک برنامه را نشان نمی‌دهد. برای نمونه، ساخت گراف فراخوانی^۱ کامل یا تحلیل جریان داده به پردازش‌های بیشتری نیاز دارد. CodeCompass از AST برای استخراج ساختار قابل اعتماد استفاده می‌کند، اما ادعای انجام تحلیل کامل جریان کنترل، جریان داده یا گراف فراخوانی را ندارد.



شکل ۲-۱: نمونه تبدیل کد پایتون به ساختار AST

^۱ Call Graph

۲-۴- قطعه‌بندی ساختار آگاه کد

مدل‌های بازیابی و تولید معمولاً نمی‌توانند تمام یک مخزن نرم‌افزاری را به‌عنوان یک ورودی واحد پردازش کنند. حتی اگر محدودیت طول ورودی وجود نداشته باشد، ورود حجم زیادی از کد نامرتب می‌تواند تشخیص شواهد اصلی را دشوار کند. بنابراین، پیش از بردارسازی و بازیابی، کد به واحدهای کوچک‌تر تقسیم می‌شود. این فرایند قطعه‌بندی نام دارد.

در قطعه‌بندی ساده، متن بر اساس تعداد ثابتی از نویسه‌ها یا خطوط بریده می‌شود. این روش ممکن است مرز یک تابع یا کلاس را از بین ببرد و بخشی از تعریف را در یک قطعه و ادامه آن را در قطعه دیگر قرار دهد. در مقابل، قطعه‌بندی ساختار آگاه از ساختار نحوی برنامه برای تعیین مرز قطعه‌ها استفاده می‌کند. در این حالت، یک تابع، متد یا نمای کلی کلاس به‌عنوان یک واحد مستقل در نظر گرفته می‌شود.

یک قطعه کد مناسب برای بازیابی تنها شامل متن خام کد نیست. افزودن اطلاعاتی مانند مسیر نسبی فایل، نوع نماد، نام کامل، کلاس والد، پارامترها، نوع بازگشتی، تزئین‌گرها و رشته‌ی مستندات¹ می‌تواند نمایش غنی‌تری از واحد کد ایجاد کند. با این حال، این اطلاعات باید از تحلیل قطعی برنامه به دست آیند و نباید توسط مدل زبانی حدس زده شوند.

اندازه قطعه یک مصالحه میان کامل بودن زمینه و دقت بازیابی ایجاد می‌کند. قطعه بسیار کوچک ممکن است اطلاعات لازم برای فهم رفتار را نداشته باشد، در حالی که قطعه بسیار بزرگ می‌تواند چند رفتار متفاوت را ترکیب کند. استفاده از مرز توابع، متدها و کلاس‌ها راهکاری طبیعی برای کنترل این مسئله در پروژه‌های پایتون است.

۲-۵- نمایش برداری و تعبیه‌سازی

نمایش برداری معنایی یا تعبیه‌سازی روشی برای تبدیل متن، کد یا ترکیبی از آن‌ها به یک بردار عددی با ابعاد ثابت است. هدف این نمایش آن است که ورودی‌های دارای معنای نزدیک در فضای برداری در موقعیت‌های نزدیک‌تری قرار گیرند. بنابراین، پرسش کاربر و قطعه‌های کد را می‌توان با یک مدل تعبیه‌سازی به بردار تبدیل و سپس میزان نزدیکی آن‌ها را محاسبه کرد.

اگر بردار پرسش را با q و بردار یک قطعه کد را با d نشان دهیم، بازیابی معنایی قطعه‌هایی را جست‌وجو می‌کند که بردارشان بیشترین شباهت را با q داشته باشد. مزیت این رویکرد آن است که وجود واژه‌های کاملاً یکسان شرط ضروری بازیابی نیست. برای مثال، یک مدل مناسب ممکن است میان عبارت فارسی «اعتبارسنجی کاربر» و تابعی با نام `validate_credentials` ارتباط معنایی برقرار کند.

¹ Docstring

کیفیت این فرایند به مدل تعبیه‌سازی وابسته است. ابعاد بیشتر بردار لزوماً به معنای کیفیت بهتر نیست؛ زیرا مدل باید بتواند ارتباط میان زبان طبیعی، زبان برنامه‌نویسی و زبان پرسش را به‌درستی بازنمایی کند. به همین دلیل، هویت ارائه‌دهنده¹، نام مدل و تعداد ابعاد بردار بخشی از مشخصات یک نمایه برداری محسوب می‌شوند. اگر یک مخزن با یک مدل خاص نمایه‌سازی شده باشد، پرسش معنایی نیز باید با پیکربندی سازگار اجرا شود.

مدل CodeBERT با هدف یادگیری نمایش مشترک برای زبان طبیعی و زبان برنامه‌نویسی ارائه شده است و کاربردهایی مانند جست‌وجوی کد و تولید مستندات را پوشش می‌دهد [4]. GraphCodeBERT نیز علاوه بر توالی توکن‌ها²، اطلاعات جریان داده را وارد فرایند پیش‌آموزش می‌کند و اهمیت ساختار کد را در یادگیری نمایش‌های برنامه نشان می‌دهد [5]. CodeCompass این مدل‌ها را آموزش یا تنظیم دقیق نمی‌کند، بلکه از مدل‌های تعبیه‌سازی موجود در یک خط لوله بازیابی قابل تعویض استفاده می‌کند. UniXcoder نیز از محتوای چندوجهی مانند AST و توضیحات کد برای تقویت نمایش قطعه استفاده می‌کند [6].

۲-۶- معیارهای شباهت برداری

پس از تولید بردار پرسش و بردار قطعه‌های کد، برای رتبه‌بندی نتایج به یک معیار فاصله یا شباهت نیاز است. معیارهای متداول شامل شباهت کسینوسی، فاصله اقلیدسی و ضرب داخلی هستند. Chroma نیز از معیارهای cosine² و ip پشتیبانی می‌کند [7].

۲-۶-۱- شباهت کسینوسی

شباهت کسینوسی زاویه میان دو بردار را مقایسه می‌کند. فرمول آن به‌صورت زیر است:

$$\text{cosine}(q,d) = (q \cdot d) / (\|q\| \|d\|)$$

در این معیار، جهت بردارها اهمیت دارد و اثر اندازه آن‌ها با تقسیم بر طول بردار حذف می‌شود. هرچه زاویه میان دو بردار کمتر باشد، شباهت آن‌ها بیشتر است. در سامانه‌هایی مانند Chroma معمولاً فاصله کسینوسی گزارش می‌شود:

$$d_cosine(q,d) = 1 - \text{cosine}(q,d)$$

بنابراین، فاصله کمتر نشان‌دهنده شباهت بیشتر است. در CodeCompass امتیاز نتیجه معنایی برای معیار کسینوسی از رابطه distance - 1 محاسبه می‌شود.

انتخاب شباهت کسینوسی در CodeCompass به این دلیل است که هدف اصلی، مقایسه جهت معنایی بردار پرسش و کد است، نه بزرگی عددی آن‌ها. این معیار در بازیابی متون و تعبیه‌سازی‌ها متداول است و

¹ Provider

² Token

Chroma نیز آن را مستقیماً پشتیبانی می‌کند. با این حال، این انتخاب به معنای اثبات برتری عمومی شباهت کسینوسی بر تمام معیارهای دیگر نیست؛ زیرا CodeCompass تاکنون آزمایش مستقلی برای مقایسه تجربی شباهت کسینوسی، فاصله اقلیدسی L2 و ضرب داخلی انجام نداده است.

۲-۶-۲- فاصله اقلیدسی

فاصله اقلیدسی یا L2 فاصله مستقیم میان دو نقطه در فضای برداری را اندازه‌گیری می‌کند:

$$d_{L2}(q,d) = \sqrt{(\sum_{i=1}^n (q_i - d_i)^2)}$$

هرچه این مقدار کمتر باشد، دو بردار به یکدیگر نزدیک‌تر هستند. برخلاف شباهت کسینوسی، فاصله اقلیدسی می‌تواند به اندازه بردارها حساس باشد. اگر بردارها به طول واحد نرمال شده باشند، رتبه‌بندی حاصل از فاصله اقلیدسی و شباهت کسینوسی ارتباط بسیار نزدیکی خواهد داشت:

$$\|q - d\|^2 = 2(1 - \text{cosine}(q,d))$$

۲-۶-۳- ضرب داخلی

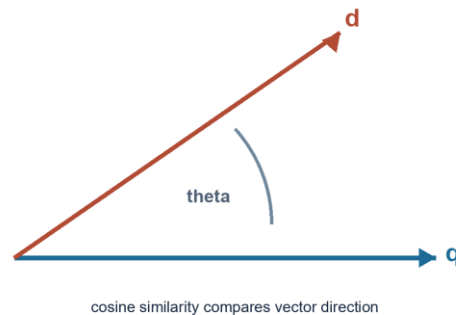
ضرب داخلی از رابطه زیر محاسبه می‌شود:

$$s_{ip}(q,d) = q \cdot d = \sum_{i=1}^n q_i d_i$$

در این روش، هم جهت و هم اندازه بردار می‌توانند بر امتیاز اثر بگذارند. در صورتی که بردارها نرمال شده باشند، ضرب داخلی و شباهت کسینوسی نتایج مشابهی ایجاد می‌کنند. با این حال، رفتار دقیق هر معیار به ویژگی‌های مدل تعبیه‌سازی و نحوه آموزش آن بستگی دارد.

۲-۶-۴- معیارهای دیگر

فاصله منهتن، Jaccard و معیارهای مبتنی بر هم‌پوشانی توکن‌ها نیز قابل استفاده‌اند. فاصله منهتن مجموع اختلاف مطلق مؤلفه‌های دو بردار را محاسبه می‌کند. معیار Jaccard بیشتر برای مقایسه مجموعه‌ها، مانند مجموعه توکن‌های پرسش و کد، مناسب است. این معیارها در پیاده‌سازی فعلی بازیابی معنایی CodeCompass استفاده نشده‌اند.



شکل ۲-۲: مفهوم زاویه میان بردار پرسش و قطعه کد در شباهت کسینوسی

۷-۲- روش‌های بازیابی اطلاعات

۲-۷-۱- بازیابی واژگانی

بازیابی واژگانی بر حضور کلمات یا توکن‌های مشترک میان پرسش و اسناد تکیه دارد. در زمینه کد، نام تابع، نام کلاس، مسیر فایل، رشته‌ی مستندات و متن کد می‌توانند برای محاسبه امتیاز استفاده شوند. این روش برای پرسش‌هایی که شامل نام دقیق نماد هستند مؤثر و قابل تفسیر است.

مزیت مهم بازیابی واژگانی آن است که به مدل تعبیه‌سازی یا نمایه برداری وابسته نیست. همچنین، اگر کاربر نام دقیق یک تابع مانند `create_project` را وارد کند، انطباق مستقیم می‌تواند نتیجه مرتبط را با اطمینان مناسبی پیدا کند. محدودیت آن زمانی آشکار می‌شود که زبان پرسش با شناسه‌های کد متفاوت باشد؛ به‌ویژه هنگامی که پرسش فارسی و کد انگلیسی است.

۲-۷-۲- بازیابی معنایی

در بازیابی معنایی، پرسش و قطعه‌های کد به بردار تبدیل می‌شوند و نتایج با استفاده از یک معیار شباهت رتبه‌بندی می‌شوند. این روش قادر است ارتباط‌هایی را تشخیص دهد که الزاماً شامل واژه مشترک نیستند. در مقابل، کیفیت آن به مدل تعبیه‌سازی، داده‌ای که مدل با آن آموزش دیده و سازگاری مدل با زبان پرسش و کد وابسته است. مدل‌های ویژه جست‌وجوی کد نیز کارایی نمایش‌های برداری را برای یافتن شواهد مرتبط بررسی کرده‌اند [8].

بازیابی معنایی ممکن است برای برخی پرسش‌ها نتایجی با مفهوم نزدیک اما از نظر وظیفه نرم‌افزاری نامرتبب بازیابی کند. همچنین، نام‌های عمومی مانند `load`، `save` یا `run` می‌توانند در قسمت‌های مختلف یک پروژه تکرار شوند. از این‌رو، استفاده از اطلاعات ساختاری در متن تعبیه‌سازی و کنترل دقیق هویت نماد اهمیت دارد.

۲-۷-۳- بازیابی ترکیبی

بازیابی ترکیبی نتایج روش واژگانی و معنایی را با یکدیگر ادغام می‌کند. هدف این روش استفاده همزمان از قدرت تطبیق دقیق شناسه‌ها و توانایی کشف ارتباط معنایی است. برای مثال، بازیابی واژگانی ممکن است در پرسش‌هایی که نام دقیق تابع را دارند بهتر عمل کند، در حالی که بازیابی معنایی می‌تواند برای پرسش‌های توصیفی فارسی مناسب‌تر باشد.

یکی از روش‌های ادغام رتبه‌ها، RRF^1 است. این روش به امتیازهای خام وابسته نیست و جایگاه هر نتیجه در فهرست‌های ورودی را در نظر می‌گیرد. امتیاز یک سند از رابطه زیر به دست می‌آید:

$$RRF(d) = \sum_{r \in R} 1 / (k + rank_r(d))$$

در این رابطه، R مجموعه رتبه‌بندی‌های ورودی، $rank_r(d)$ جایگاه سند d در رتبه‌بندی r و k یک ثابت کنترلی است. پژوهش اصلی RRF این روش را به‌عنوان راهکاری ساده و بدون نیاز به داده آموزشی برای ترکیب چند رتبه‌بندی معرفی کرده است [9].

در $CodeCompass$ ، رتبه‌های روش واژگانی و معنایی با وزن برابر و مقدار $k=60$ ادغام می‌شوند. استفاده از رتبه به‌جای امتیاز خام اهمیت دارد؛ زیرا امتیاز روش واژگانی و شباهت کسینوسی در مقیاس یکسانی قرار ندارند و جمع مستقیم آن‌ها قابل اتکا نیست. مقایسه روش‌های بازیابی مورد استفاده در $CodeCompass$ در جدول ۱-۲ ارائه شده است.

جدول ۱-۲: مقایسه روش‌های بازیابی مورد استفاده در $CodeCompass$

روش	مبنای بازیابی	مزیت اصلی	محدودیت اصلی
واژگانی	اشتراک توکن و شناسه	مناسب برای نام‌های دقیق و قابل تفسیر	ضعف در تطبیق پرسش فارسی با شناسه انگلیسی
معنایی	تعبیه‌سازی و شباهت برداری	بازیابی بر اساس مفهوم	وابستگی به مدل تعبیه‌سازی و کیفیت نمایه
ترکیبی	ادغام رتبه‌های واژگانی و معنایی	استفاده از نقاط قوت هر دو روش	هزینه بیشتر و امکان جابه‌جایی برخی نتایج مناسب

¹ Reciprocal Rank Fusion (RRF)

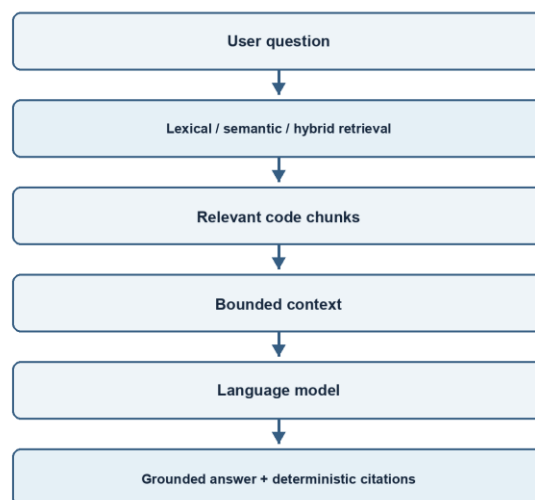
۲-۸- تولید تقویت‌شده با بازیابی

معماری RAG از دو بخش اصلی بازیابی و تولید تشکیل می‌شود. ابتدا پرسش کاربر برای بازیابی اطلاعات مرتبط استفاده می‌شود. سپس شواهد بازیابی‌شده به‌همراه پرسش در اختیار مدل زبانی قرار می‌گیرند و مدل پاسخ را با اتکا به این زمینه تولید می‌کند. در پژوهش اصلی RAG، این معماری به‌عنوان ترکیب حافظه پارامتری مدل و حافظه غیرپارامتری قابل بازیابی معرفی شده است [2].

در کاربردهای مربوط به کد، منبع اطلاعاتی به‌جای اسناد عمومی از قطعه‌های یک مخزن نرم‌افزاری تشکیل می‌شود. پرسش کاربر ابتدا به بخش بازیابی ارسال شده و قطعه‌های مرتبط شناسایی می‌شوند. سپس تنها تعداد محدودی از قطعه‌های رتبه‌بالا، با رعایت بودجه زمینه، وارد دستور مدل می‌شوند. این محدودسازی دو مزیت دارد: نخست، حجم ورودی مدل کاهش می‌یابد و دوم، احتمال اثرگذاری کدهای نامرتبط بر پاسخ کمتر می‌شود. رویکردهای سطح مخزن مانند RepoCoder نیز از بازیابی برای انتخاب اطلاعات پراکنده میان فایل‌ها پیش از تولید استفاده می‌کنند [10].

وجود مرحله بازیابی به‌تنهایی تضمین نمی‌کند که پاسخ نهایی صحیح باشد. اگر قطعه مرتبط بازیابی نشود، مدل نمی‌تواند رفتار واقعی را تنها از روی پرسش تشخیص دهد. همچنین، اگر زمینه شامل چند نماد مشابه باشد، مدل ممکن است رفتار آن‌ها را با یکدیگر ترکیب کند. بنابراین، کیفیت سامانه RAG حاصل عملکرد مشترک بازیابی، انتخاب زمینه، دستور مدل، مدل زبانی و اعتبارسنجی خروجی است.

در CodeCompass، زمینه شامل کد بازیابی‌شده و اطلاعات قطعی مربوط به فایل، نماد و محدوده خطوط است. مدل تنها وظیفه تولید توضیح طبیعی را بر عهده دارد. مسیر فایل و شماره خطوط از متن تولیدشده توسط مدل استخراج نمی‌شوند؛ بلکه از فراداده ذخیره‌شده سامانه به پاسخ افزوده می‌شوند.



شکل ۲-۳: فرایند عمومی پاسخ‌گویی مبتنی بر RAG در یک سامانه درک کد

۲-۹- مدل‌های زبانی و تولید توضیح کد

مدل زبانی بزرگ یا LLM¹ مدلی است که با استفاده از حجم زیادی از داده‌های متنی یا کد آموزش می‌بیند و می‌تواند بر اساس ورودی، متن جدید تولید کند. در سامانه‌های درک کد، مدل زبانی می‌تواند برای توضیح عملکرد تابع، پاسخ به پرسش کاربر یا تولید مستندات اولیه استفاده شود.

مدل‌های زبانی نباید به‌عنوان منبع قطعی اطلاعات ساختاری پروژه در نظر گرفته شوند. نام فایل، نام کامل نماد، محدوده خطوط، پارامترهای واقعی و مسیر منبع باید از تحلیل کد یا پایگاه داده معتبر به دست آیند. اگر این اطلاعات به مدل سپرده شوند، امکان تولید مسیر یا مشخصات غیرواقعی وجود دارد.

در CodeCompass مسئولیت‌ها به دو دسته تقسیم می‌شوند. داده‌هایی که باید قابل اعتماد و قابل بررسی باشند، مانند هویت نماد و ارجاع، به‌صورت قطعی توسط سامانه تولید می‌شوند. در مقابل، وظیفه مدل به نگارش توضیح طبیعی بر اساس شواهد محدود می‌شود. اگر شواهد کافی در دسترس نباشد، سامانه باید به‌جای حدس‌زدن، کمبود شواهد را اعلام کند.

در حوزه مستندسازی کد نیز استفاده از زمینه پروژه اهمیت دارد. پژوهش ProConSuL با استفاده از اطلاعات ساختاری و زمینه مرتبط پروژه نشان داده است که ارائه زمینه دقیق می‌تواند کیفیت خلاصه‌سازی کد را افزایش و توهم مدل را کاهش دهد [11]. با این حال، این پژوهش از گراف فراخوانی، آموزش اختصاصی و تمرکز بر پروژه‌های C و ++C استفاده می‌کند؛ در حالی که CodeCompass یک سامانه پرسش‌محور برای پروژه‌های پایتون است و مدل جدیدی آموزش نمی‌دهد.

۲-۱۰- پرسش‌های فارسی درباره کد انگلیسی

در بسیاری از پروژه‌های نرم‌افزاری، نام فایل‌ها، کلاس‌ها و توابع به زبان انگلیسی نوشته می‌شوند؛ اما کاربر ممکن است پرسش خود را به زبان فارسی مطرح کند. در این حالت، بازیابی واژگانی با چالش آشکاری روبه‌رو است، زیرا ممکن است هیچ توکن مشترکی میان پرسش فارسی و کد انگلیسی وجود نداشته باشد.

تعبیه‌سازی چندزبانه می‌تواند پرسش فارسی و کد انگلیسی را به فضای برداری مشترکی نگاشت کند. موفقیت این فرایند به توانایی مدل در نمایش ارتباط میان دو زبان و مفاهیم برنامه‌نویسی وابسته است. به همین دلیل نمی‌توان فرض کرد هر مدل تعبیه‌سازی برای این مسئله عملکرد یکسانی دارد. پژوهش درباره مجموعه داده‌های چندزبانه جست‌وجوی کد نیز نشان می‌دهد زبان پرسش و کیفیت داده چندزبانه بر عملکرد بازیابی اثر می‌گذارند [12].

متن فارسی چالش‌های دیگری نیز ایجاد می‌کند؛ از جمله تفاوت نویسه‌های عربی و فارسی، وجود یا نبود

¹ Large Language Model (LLM)

نیم‌فاصله، شکل‌های متفاوت نوشتن جمع‌ها و ترکیب واژه فارسی با شناسه انگلیسی. نرمال‌سازی متن می‌تواند بخشی از این تفاوت‌ها را کاهش دهد، اما نباید باعث تغییر شناسه‌های واقعی کد شود. بنابراین، پردازش پرسش طبیعی و حفظ دقیق نام‌های برنامه‌نویسی باید از یکدیگر تفکیک شوند.

CodeCompass از هر سه روش بازیابی واژگانی، معنایی و ترکیبی پشتیبانی می‌کند. این طراحی امکان مقایسه تجربی روش‌ها را فراهم کرده و وابستگی سامانه به یک روش واحد را کاهش می‌دهد. نتایج کمی این مقایسه در فصل ارزیابی ارائه خواهند شد.

۲-۱۱- پژوهش‌های مرتبط

۲-۱۱-۱ CodeSearchNet

CodeSearchNet یک مجموعه داده و چالش برای ارزیابی جست‌وجوی معنایی کد ارائه می‌کند. در این پژوهش، پرسش‌های زبان طبیعی برای یافتن توابع مرتبط استفاده شده‌اند و ارزیابی انسانی نیز برای تعیین میزان ارتباط نتایج به کار رفته است [1]. این کار بیشتر بر مجموعه داده، روش ارزیابی و بازیابی کد تمرکز دارد و یک سامانه کامل برای پاسخ‌گویی مستند درباره مخزن محلی ارائه نمی‌کند.

۲-۱۱-۲ CodeBERT

CodeBERT یک مدل از پیش‌آموزش دیده دومی‌داده برای زبان طبیعی و زبان برنامه‌نویسی است. این مدل با هدف یادگیری نمایش‌هایی طراحی شده است که بتوانند در وظایفی مانند جست‌وجوی کد و تولید مستندات استفاده شوند [4]. تفاوت CodeCompass با CodeBERT در آن است که CodeCompass مدل جدیدی آموزش نمی‌دهد و تمرکز آن بر طراحی خط لوله بازیابی، تولید پاسخ و ارجاع قابل بررسی است.

۲-۱۱-۳ GraphCodeBERT

GraphCodeBERT با وارد کردن اطلاعات جریان داده به فرایند پیش‌آموزش، ساختار برنامه را فراتر از توالی توکن‌ها در نظر می‌گیرد [5]. این پژوهش اهمیت اطلاعات ساختاری را در نمایش کد نشان می‌دهد. CodeCompass به جای آموزش مدل ساختار آگاه، از AST برای استخراج قطعی واحدهای کد و تولید قطعه‌های ساختاری استفاده می‌کند.

۲-۱۱-۴ ProConSuL

ProConSuL چارچوبی برای خلاصه‌سازی کد با استفاده از زمینه پروژه و اطلاعات ساختاری ارائه می‌دهد. این روش با ساخت گراف فراخوانی و استفاده از اطلاعات توابع فراخوانی شده، زمینه دقیق‌تری در اختیار مدل قرار می‌دهد [11]. CodeCompass برخلاف این روش بر پرسش کاربر، بازیابی قطعه‌های مرتبط، پشتیبانی از زبان فارسی و ارجاع قطعی تمرکز دارد و گراف فراخوانی کامل تولید نمی‌کند.

۲-۱۱-۵- تولید تقویت‌شده با بازیابی

پژوهش Lewis و همکاران، معماری RAG را به‌عنوان ترکیب مدل مولد و حافظه خارجی قابل بازیابی مطرح کرده است [2]. CodeCompass این ایده را برای پایگاه کد پایتون به کار می‌گیرد؛ به‌گونه‌ای که حافظه خارجی از قطعه‌های ساختاری کد تشکیل شده و خروجی به محل دقیق شواهد متصل می‌شود. مقایسه CodeCompass با پژوهش‌های مرتبط در جدول ۲-۲ ارائه شده است.

جدول ۲-۲: مقایسه CodeCompass با پژوهش‌های مرتبط

پژوهش	تمرکز اصلی	استفاده از ساختار کد	خروجی پرسش‌محور	پشتیبانی فارسی	ارجاع قطعی منبع
CodeSearchNet	ارزیابی جست‌وجوی معنایی کد	محدود	خیر	خیر	خیر
CodeBERT	یادگیری نمایش مشترک متن و کد	محدود	خیر	خیر	خیر
GraphCodeBERT	نمایش کد با جریان داده	بله	خیر	خیر	خیر
ProConSuL	خلاصه‌سازی با زمینه پروژه	بله	خیر	خیر	خیر
CodeCompass	بازیابی و توضیح کد پایتون	بله، مبتنی بر AST	بله	بله	بله

۲-۱۲- جایگاه CodeCompass و شکاف پژوهشی

بررسی کارهای پیشین نشان می‌دهد هرکدام بخشی از مسئله درک کد را پوشش داده‌اند. CodeSearchNet بر بازیابی معنایی و ارزیابی آن تمرکز دارد. CodeBERT و GraphCodeBERT به یادگیری نمایش‌های مناسب برای زبان طبیعی و کد می‌پردازند. ProConSuL اهمیت زمینه پروژه و ساختار برنامه را برای خلاصه‌سازی کد بررسی می‌کند. معماری RAG نیز چارچوبی عمومی برای ترکیب بازیابی و تولید پاسخ فراهم می‌سازد.

CodeCompass الگوریتم جدیدی برای تعبیه‌سازی یا معیار شباهت ارائه نمی‌کند. سهم اصلی آن در ترکیب مهندسی‌شده این مفاهیم برای ساخت یک سامانه دوزبانه و قابل بررسی است. این سامانه مخزن پایتون را به‌صورت قطعی تحلیل می‌کند، واحدهای ساختاری کد را ایجاد می‌کند، امکان مقایسه بازیابی واژگانی، معنایی

و ترکیبی را فراهم می‌سازد و پاسخ مدل را به ارجاع‌هایی متصل می‌کند که از فراداده معتبر سامانه ساخته شده‌اند.

تمایز مهم دیگر، جداسازی مسئولیت مدل از مسئولیت سامانه است. مدل زبانی می‌تواند توضیح طبیعی تولید کند، اما اجازه تعیین هویت منبع، مسیر فایل یا محدوده خطوط را ندارد. این مرزبندی سبب می‌شود کاربر بتواند پاسخ را مستقیماً با کد منبع مقایسه کند. افزون بر این، CodeCompass برای پرسش‌های فارسی و انگلیسی طراحی و بر مجموعه‌ای ثابت از پرسش‌ها و مخازن ارزیابی شده است. روش و نتایج این ارزیابی در فصل چهارم تشریح خواهند شد.

۲-۱۳- جمع‌بندی

در این فصل، مفاهیم نظری موردنیاز برای طراحی CodeCompass بررسی شدند. جست‌وجوی واژگانی برای یافتن انطباق‌های مستقیم مناسب است، اما در پرسش‌هایی که واژگان کاربر با شناسه‌های کد متفاوت‌اند محدودیت دارد. بازیابی معنایی با تبدیل پرسش و قطعه‌های کد به بردار، امکان مقایسه مفهومی آن‌ها را فراهم می‌کند. شباهت کسینوسی نیز جهت بردارها را مستقل از اندازه آن‌ها مقایسه می‌کند و معیار مورد استفاده در نمایه معنایی CodeCompass است؛ هرچند برتری آن نسبت به سایر معیارها در این پروژه به‌صورت مستقل آزمایش نشده است.

همچنین، AST به‌عنوان مبنایی برای استخراج قطعی ساختار برنامه و قطعه‌بندی بر اساس مرز کلاس‌ها، توابع و متدها معرفی شد. ترکیب روش واژگانی و معنایی با RRF امکان استفاده هم‌زمان از انطباق دقیق و ارتباط مفهومی را فراهم می‌کند. در معماری RAG، قطعه‌های بازیابی‌شده به‌عنوان زمینه در اختیار مدل زبانی قرار می‌گیرند تا پاسخ به محتوای واقعی مخزن متکی باشد.

در پایان، پژوهش‌های CodeSearchNet، CodeBERT، GraphCodeBERT، ProConSuL و RAG بررسی شدند. جایگاه CodeCompass در ترکیب تحلیل ساختاری کد، بازیابی دوزبانه، تولید توضیح مبتنی بر شواهد و ارجاع قطعی مشخص شد. در فصل بعد، معماری، اجزای اصلی و نحوه پیاده‌سازی این سامانه تشریح می‌شوند.

فصل سوم

طراحی و پیاده‌سازی سامانه CODECOMPASS

۳-۱- مقدمه

در فصل پیش، مبانی تحلیل ساختاری کد، قطعه‌بندی ساختارآگاه، بازیابی واژگانی و معنایی و معماری RAG بررسی شد. در این فصل، نحوه تبدیل آن مفاهیم به سامانه نهایی CodeCompass تشریح می‌شود. تمرکز فصل بر تصمیم‌هایی است که قابلیت ردیابی پاسخ تا کد منبع را حفظ می‌کنند: مدل می‌تواند در بازیابی و تولید متن مشارکت کند، اما هویت فایل، نام نماد، محدوده خطوط، شناسه قطعه و ارجاع از فراداده قطعی سامانه به دست می‌آید.

CodeCompass یک سامانه محلی‌محور برای مطالعه مخزن‌های پایتون است. کاربر مسیر یک مخزن را انتخاب می‌کند؛ سامانه فایل‌های مجاز را به‌صورت امن پویش می‌کند، ساختار نحوی را با AST استخراج می‌نماید، قطعه‌های معنادار می‌سازد و فراداده را در SQLite نگه می‌دارد. سپس با بازیابی واژگانی، معنایی یا ترکیبی، شواهد مناسب برای پرسش فارسی یا انگلیسی انتخاب می‌شوند. پاسخ طبیعی توسط مدل زبانی تولید می‌شود، ولی ارجاع به مسیر، نماد و خطوط مستقل از متن مدل ساخته و در رابط کاربری نمایش داده می‌شود. این فصل ابتدا نیازمندی‌ها و محدوده سامانه را مشخص می‌کند، سپس معماری و جریان‌های اصلی را توضیح می‌دهد. در ادامه پویش مخزن، تحلیل AST، قطعه‌بندی، ذخیره‌سازی، نمایه‌سازی، بازیابی، ساخت زمینه، پاسخ‌گویی مستند، مستندسازی تابع، ارائه‌دهنده‌ها، API¹ و رابط کاربری بررسی می‌شوند. در پایان نیز ملاحظات امنیت، حریم خصوصی و مدیریت خطا جمع‌بندی خواهد شد.

۳-۲- نیازمندی‌های سامانه

نیازمندی‌ها از تعهدهای پروژه و رفتار نسخه نهایی استخراج شده‌اند. تفکیک نیازمندی‌های کارکردی و غیرکارکردی کمک می‌کند قابلیت‌های قابل مشاهده از ویژگی‌های کیفیتی مانند قابلیت اعتماد، امنیت و

¹ Application Programming Interface (API)

تکرارپذیری جدا شوند.

۳-۲-۱- نیازمندی‌های کارکردی

نیازمندی‌های کارکردی در جدول ۳-۱ خلاصه شده‌اند. محور مشترک آن‌ها این است که مسیر کامل از انتخاب مخزن تا مشاهده پاسخ و کد مورد استناد در یک گردش کار واحد در دسترس باشد.

جدول ۳-۱: نیازمندی‌های کارکردی CodeCompass

شناسه	نیازمندی	خروجی قابل مشاهده
F1	انتخاب و ثبت یک مخزن محلی پایتون	پروژه و وضعیت نمایه‌سازی
F2	پویش امن فایل‌های مجاز و استخراج ساختار AST	فایل‌ها، کلاس‌ها، توابع، متدها و محدوده خطوط
F3	ساخت و نگهداری نمایه اولیه و افزایشی	فراداده SQLite و بردارهای ChromaDB
F4	جست‌وجوی واژگانی، معنایی و ترکیبی	قطعه‌های رتبه‌بندی‌شده با ارجاع
F5	پاسخ‌گویی به پرسش فارسی یا انگلیسی بر اساس شواهد	پاسخ طبیعی یا اعلام شواهد ناکافی
F6	تولید مستندات تابع یا متد منتخب	حقایق قطعی و توضیح مدل با ارجاع
F7	مرور فایل‌ها و نمادها و بازکردن محل ارجاع	نمایش کد و برجسته‌سازی خطوط در Monaco
F8	نمایش نتایج ارزیابی نهایی	داشبورد معیارها و جزئیات اجراهای منجمد

۳-۲-۲- نیازمندی‌های غیرکارکردی

کیفیت پاسخ تنها به روانی متن وابسته نیست. سامانه باید بتواند منشأ هر ادعای کدمحور را نشان دهد، در شکست نمایه‌سازی داده معتبر قبلی را حفظ کند و بدون وابستگی اجباری به سرویس شبکه اجرا شود. جدول ۳-۲ مهم‌ترین نیازمندی‌های غیرکارکردی را نشان می‌دهد.

جدول ۳-۲: نیازمندی‌های غیرکارکردی CodeCompass

ویژگی	معیار طراحی
قابلیت اعتماد	اطلاعات هویتی و ارجاع فقط از فراداده قطعی ساخته شود.
ایمنی نمایه	نمایه ناقص فعال نشود و در شکست کنترل شده نمایه معتبر قبلی باقی بماند.
تکرارپذیری	هویت ارائه‌دهنده، مدل، ابعاد بردار و نسخه طرح نمایه ثبت و پیش از بازیابی کنترل شود.
حریم خصوصی	اجرای کاملاً محلی با Ollama ممکن باشد و کلیدها یا اسرار در داده پروژه ذخیره نشوند.
کارایی	در تغییرهای محدود فقط فایل‌های تغییر کرده دوباره تحلیل و بردارسازی شوند.
بازیابی پذیری خطا	خطاهای فایل، نحو، ارائه‌دهنده و ناسازگاری نمایه به پیام کنترل شده تبدیل شوند.
قابلیت استفاده	کاربر بتواند از نتیجه جست‌وجو یا پاسخ مستقیماً به خطوط منبع منتقل شود.

۳-۲-۳- ملاحظات کارایی و مقیاس‌پذیری

برای کنترل هزینه و زمان در مخزن‌های بزرگ‌تر، نمایه‌سازی افزایشی فقط فایل‌های افزوده یا تغییر کرده را دوباره تحلیل و بردارسازی می‌کند و بردار قطعه‌های بدون تغییر، پس از کنترل هویت مدل، ابعاد و هش محتوا، دوباره به کار می‌رود. بردارسازی به صورت دسته‌ای انجام می‌شود، تعداد نتایج و اندازه زمینه کران‌دار است و ساخت نمایه‌ی نامزد^۱ مانع فعال شدن نمایه ناقص می‌شود. جداسازی SQLite به عنوان منبع اصلی حقیقت از ChromaDB به عنوان نمایه قابل جایگزینی نیز بازسازی و بازیابی پس از خطا را ساده می‌کند. حدود تجربی اندازه مخزن، زمان نمایه‌سازی و تأخیر بازیابی در فصل ارزیابی گزارش می‌شوند.

۳-۳- محدوده سامانه و موارد خارج از محدوده

محدوده نهایی CodeCompass تحلیل مخزن‌های کوچک و متوسط پایتون، بازیابی شواهد مرتبط و تولید توضیح مستند برای مطالعه کد است. سامانه فایل‌های پایتون را تحلیل می‌کند، نمادهای اصلی را استخراج می‌نماید، جست‌وجو و پرسش و پاسخ دوزبانه ارائه می‌دهد و برای تابع یا متد منتخب مستندات هدفمند تولید می‌کند. رابط وب، API محلی و داشبوردهای ارزیابی نیز جزو خروجی تحویل شده‌اند.

موارد خارج از محدوده شامل آموزش یا تنظیم دقیق مدل، تحلیل زبان‌های برنامه‌نویسی دیگر، ساخت گراف فراخوانی کامل، تحلیل ایستای جامع، ویرایش خودکار کد، احراز هویت و چندکاربری، استقرار ابری،

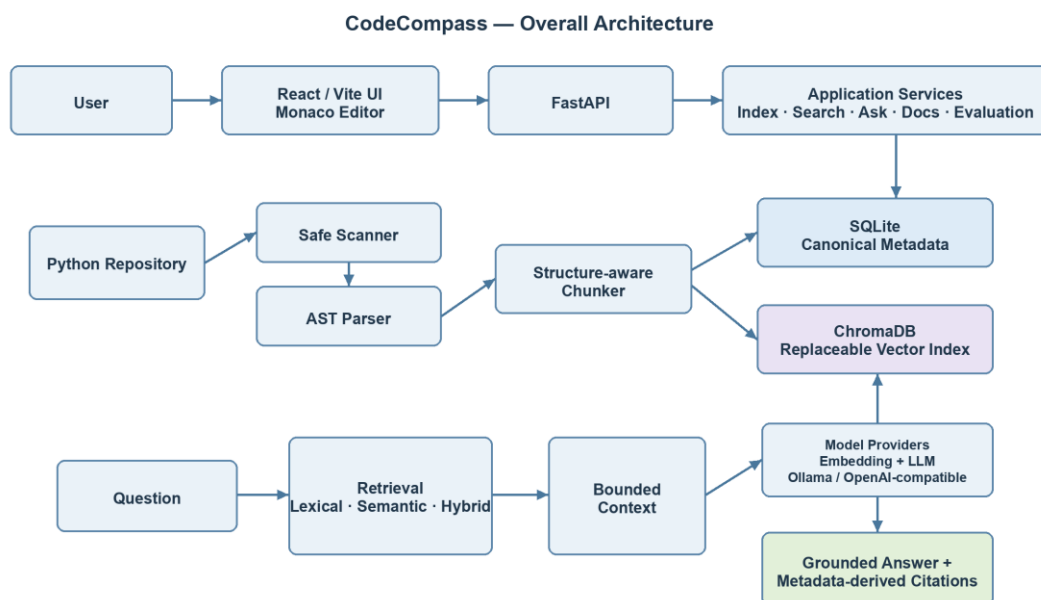
¹ Candidate Index

دریافت مخزن از GitHub و افزونه¹ IDE است. تماس‌های مستقیم تابع که در مستندسازی نمایش داده می‌شوند با پیمایش محلی AST استخراج می‌گردند و نباید با گراف فراخوانی کامل بین‌فایلی اشتباه شوند. این مرزبندی مانع آن است که قابلیت‌های پژوهشی پیشنهادی اما پیاده‌نشده به‌عنوان خروجی نهایی گزارش شوند.

طراحی فعلی عمدتاً برای پرسش‌های محلی و نمادمحور، مانند یافتن پیاده‌سازی یک رفتار یا توضیح تابع و کلاس، بهینه شده است. سامانه ممکن است برای بعضی پرسش‌های سراسری مخزن شواهد مفید بازایی کند، اما نمایه‌ی مخزن، مسیرياب پرسش، گراف دانش و گردش کار عامل‌محور اختصاصی ندارد؛ از این رو پوشش تضمین‌شده‌ای برای پرسش‌های معماری کلان یا استخراج عمومی طرح پایگاه داده مخزن هدف ارائه نمی‌شود. اگر شواهد بازایی شده کافی نباشند، سامانه باید همان محدودیت را صریح اعلام کند.

۳-۴- معماری کلی CodeCompass

معماری سامانه از چند لایه صریح تشکیل شده است. لایه رابط کاربری با React و Vite ساخته شده و از طریق FastAPI با سرویس‌های کاربردی ارتباط دارد. سرویس نمایه‌سازی مخزن را به فراداده ساختاری و بردار تبدیل می‌کند؛ سرویس بازایی روی SQLite و ChromaDB عمل می‌کند؛ سرویس پاسخ‌گویی زمینه محدود را به LLM می‌دهد؛ و سرویس ارجاع، اطلاعات منبع را از پایگاه داده قطعی به خروجی متصل می‌کند. شکل ۱-۳ ارتباط این لایه‌ها را نشان می‌دهد.



شکل ۱-۳: معماری کلی سامانه CodeCompass

¹ Integrated Development Environment (IDE)

در این معماری SQLite منبع اصلی حقیقت است و وضعیت پروژه، فایل، نماد، قطعه و نسل فعال نمایه را نگه می‌دارد. ChromaDB فقط نمایه بازیابی برداری است و هر بردار با شناسه پایدار قطعه SQLite کلید می‌خورد. به همین دلیل نتیجه معنایی پیش از نمایش از SQLite بازخوانی می‌شود و مسیر یا خطوط هرگز از متن مدل یا فراداده آزاد پایگاه برداری پذیرفته نمی‌شوند.

۳-۴-۱- سهم فنی و تصمیم‌های طراحی سامانه

سهم این پروژه ابداع یک الگوریتم بازیابی یا مدل زبانی جدید نیست، بلکه طراحی، پیاده‌سازی و ارزیابی یک زنجیره یکپارچه برای فهم دوزبانه کد پایتون است. این زنجیره قطعه‌بندی ساختارآگاه مبتنی بر AST، بازیابی واژگانی و معنایی و ادغام ترکیبی، اجرای محلی محور و استقلال ارائه‌دهنده‌ها را با مرز اعتماد قطعی پیوند می‌دهد. در این مرز، مدل فقط نثر پاسخ را تولید می‌کند و شناسه قطعه، مسیر فایل، نام نماد و محدوده خطوط از SQLite و تحلیل ساختاری استخراج می‌شوند. ارزش فنی سامانه در کنار هم قرار گرفتن این تصمیم‌ها و سنجش کنترل‌شده آن‌هاست؛ نه در ادعای برتری عمومی یک مؤلفه منفرد.

۳-۵-۱- جریان کامل عملکرد سامانه

۳-۵-۱- انتخاب مخزن

کاربر مسیر ریشه یک مخزن محلی را در رابط وارد می‌کند. بخش پشتیبان^۱ مسیر را به شکل قطعی حل می‌کند و تنها در صورت وجود و پوشه‌بودن آن، فرایند را ادامه می‌دهد. پروژه با مسیر ریشه و تنظیمات لازم ثبت می‌شود؛ اما هیچ فایل خارج از همین ریشه مجاز به ورود به جریان تحلیل نیست.

۳-۵-۲- تحلیل و نمایه‌سازی

اسکنر فایل‌های پایتون مجاز را همراه با اندازه، زمان تغییر و SHA-256^۲ جمع‌آوری می‌کند. تحلیل‌گر نحوی^۳ برای هر فایل AST می‌سازد و نمادها و محدوده خطوط را استخراج می‌نماید. قطعه‌بند^۴ بر اساس مرز تابع، متد و نمای کلی کلاس قطعه می‌سازد. فراداده قطعی در SQLite و بردار متن بازیابی در یک نمایه نامزد ذخیره می‌شود؛ نمایه نامزد تنها پس از اعتبارسنجی کامل فعال خواهد شد.

۳-۵-۳- بازیابی شواهد

پرسش کاربر به یکی از سه روش واژگانی، معنایی یا ترکیبی اجرا می‌شود. هر نتیجه با chunk_id به رکورد قطعی SQLite متصل می‌گردد. سپس نتایج مرتب، تکرارها حذف و تعداد آن‌ها با limit محدود می‌شود. بدین

^۱ Backend

^۲ Secure Hash Algorithm 256-bit (SHA-256)

^۳ Parser

^۴ Chunker

ترتیب ورودی مرحله تولید نه کل مخزن، بلکه مجموعه کوچکی از قطعه‌های مرتبط و قابل ارجاع است.

۳-۵-۴- تولید پاسخ

قطعه‌های منتخب تا سقف بودجه کاراکتری در زمینه قرار می‌گیرند. دستور مدل به مدل اعلام می‌کند که کد و زمینه دستور قابل اجرا نیستند و پاسخ باید تنها بر شواهد موجود تکیه کند و به زبان پرسش نوشته شود. دمای پیش‌فرض صفر انتخاب شده است تا تغییرپذیری پاسخ کاهش یابد.

۳-۵-۵- ساخت ارجاع

پس از تولید متن، سامانه ارجاع را از مسیر فایل، نام نماد و محدوده خطوط ذخیره‌شده برای همان قطعه می‌سازد. مدل اجازه افزودن یا تغییر این اطلاعات را ندارد. بنابراین حتی اگر متن مدل ناقص باشد، مکان منبع همچنان از داده قطعی سامانه حاصل می‌شود.

۳-۵-۶- نمایش کد منبع

رابط کاربری نتیجه را همراه با ارجاع نمایش می‌دهد. با انتخاب ارجاع، محتوای فایل از API دریافت و در Monaco Editor باز می‌شود؛ سپس بازه start_line تا end_line آشکار و برجسته می‌گردد. این مرحله چرخه پرسش تا بررسی مستقیم شاهد را کامل می‌کند.

۳-۶-۶- پوشش امن مخزن

۳-۶-۱- کنترل مسیر ریشه

مرز اعتماد اسکنر از مسیر ریشه آغاز می‌شود. مسیر به‌صورت سخت‌گیرانه حل و بررسی می‌شود، باید پوشه باشد و os.walk بدون دنبال کردن پیوندهای نمادین اجرا می‌گردد. مسیر نهایی هر فایل نیز دوباره به‌شکل قطعی حل می‌شود و تنها اگر داخل ریشه باقی مانده باشد پذیرفته خواهد شد. خود فایل‌های پیوند نمادین^۱ نادیده گرفته می‌شوند تا مسیرهای بیرونی به‌صورت غیرمستقیم وارد مخزن نشوند.

۳-۶-۲- فایل‌های قابل پذیرش

نسخه نهایی تنها فایل‌های دارای پسوند py را برای تحلیل انتخاب می‌کند. برای هر فایل پذیرفته‌شده، مسیر نسبی POSIX^۲، مسیر مطلق داخلی، اندازه، mtime_ns و هش SHA-256 محاسبه می‌شود. مسیر مطلق برای عملیات محلی لازم است، اما هویت پایدار و قابل انتشار فایل بر مسیر نسبی داخل پروژه تکیه دارد.

^۱ Symlink

^۲ Portable Operating System Interface (POSIX)

۳-۶-۳- حذف فایل‌های محرمانه، محیط مجازی، حافظه‌های نهان^۱ و خروجی ساخت

پوشه‌های `.git`، `.venv`، `__pycache__`، `pytest_cache`، `mypy_cache`، `build`، `dist`، `node_modules`، `coverage`، `idea` و `vscode` از پوشش حذف می‌شوند. فایل‌های `env` و الگوی `*.env` نیز هرگز نمایه نمی‌شوند. این قواعد هم حجم داده نامرتبط را کاهش می‌دهند و هم احتمال ورود اسرار، وابستگی‌های نصب‌شده، کش‌ها و خروجی‌های ساخت به زمینه مدل را محدود می‌کنند. خطای خواندن یک فایل به صورت وضعیت کنترل‌شده ثبت می‌شود و باعث خروج بی‌قاعده کل فرایند نمی‌گردد.

۳-۷-۷- تحلیل ساختاری کد پایتون

۳-۷-۱- استخراج واحدهای ماژولی

در CodeCompass هر فایل پایتون واحد ماژولی تحلیل است؛ پیاده‌سازی نماد جداگانه‌ای با نوع `Module` ایجاد نمی‌کند. تحلیل‌گر نحوی با `tokenize.open` فایل را مطابق اعلان `Encoding` پایتون می‌خواند و با `ast.parse` درخت نحو انتزاعی را می‌سازد. دستورهای واردسازی و اعضای سطح ماژول از همین درخت استخراج می‌شوند و خطای نحوی به عنوان نتیجه قابل گزارش بازگردانده می‌شود.

۳-۷-۲- کلاس‌ها

برای هر `ClassDef` نام ساده و نام واجدشرایط، کلاس والد، محدوده شروع و پایان، کلاس‌های پایه، تزئین‌گرها و رشته‌ی مستندات استخراج می‌شود. نام واجدشرایط امکان تمایز کلاس‌های هم‌نام در دامنه‌های متفاوت را فراهم می‌کند و `parent_qualified_name` رابطه نماد تودرتو را نگه می‌دارد.

۳-۷-۳- توابع و متدها

هر `FunctionDef` و `AsyncFunctionDef` تحلیل می‌شود. اگر تابع مستقیماً در بدنه کلاس باشد به عنوان متد و در غیر این صورت به عنوان تابع ثبت می‌شود. وضعیت `async` نیز صریحاً نگهداری می‌شود. این تفکیک برای جست‌وجو، قطعه‌بندی و انتخاب هدف مستندسازی لازم است.

۳-۷-۴- امضا، پارامترها، رشته‌ی مستندات و محدوده خطوط

تحلیل‌گر نحوی پارامترهای `positional-only`، معمولی، `*args`، `keyword-only` و `**kwargs` اعلان‌های نوع^۲، مقدارهای پیش‌فرض و اعلان نوع بازگشتی را استخراج می‌کند. رشته‌ی مستندات از `AST` به دست می‌آید و `start_line` و `end_line` از گره نحوی ثبت می‌شوند. این محدوده‌ها بعداً مبنای استخراج متن دقیق قطعه و ارجاع هستند؛ از این رو مدل زبانی در تعیین آن‌ها نقشی ندارد.

^۱ Cache

^۲ Type Annotation

۳-۸- قطعه‌بندی ساختار آگاه

قطعه‌بندی بر واحدهای معنایی برنامه منطبق است. بدنه کامل تابع یا متد یک قطعه مستقل می‌سازد. برای کلاس، یک `class_overview` تولید می‌شود که تعریف و اعضای سطح کلاس را نگه می‌دارد، ولی بدنه تعریف مستقیم متدها و کلاس‌های فرزند را حذف می‌کند تا یک قطعه بسیار بزرگ و تکراری ایجاد نشود. متن تعبیه‌سازی علاوه بر کد، مسیر، نوع قطعه، نام واجدشرایط، والد، وضعیت `async`، پارامترها، نوع بازگشتی، تزئین‌گرها، کلاس‌های پایه و رشته‌ی مستندات را در بر می‌گیرد.

۳-۸-۱- تولید شناسه پایدار

`chunk_id` با SHA-256 از مسیر نسبی، نوع قطعه، نام واجدشرایط، محدوده خطوط و `content_hash` ساخته می‌شود. این شناسه در یک تصویر لحظه‌ای^۱ ثابت تکرارپذیر است و کلید اتصال `SQLite` و `ChromaDB` محسوب می‌شود. تغییر کد یا جابه‌جایی خطی که هویت قطعه را عوض کند، شناسه جدیدی تولید می‌نماید و مانع اتصال بردار قدیمی به محتوای جدید می‌شود.

۳-۸-۲- هش محتوا

`content_hash` برابر SHA-256 متن کد همان قطعه است. این مقدار برای تشخیص تغییر، کنترل استفاده مجدد از بردار و بررسی سازگاری نمایه‌ی نامزد به کار می‌رود. در سطح فایل نیز هش مستقل نگهداری می‌شود تا برنامه نمایه‌سازی افزایشی بتواند فایل‌های افزوده، تغییرکرده، حذف‌شده و بدون تغییر را تفکیک کند.

۳-۸-۳- ارتباط قطعه با فایل و نماد

هر قطعه در `SQLite` به `project_id` و `file_id` متصل است و در صورت وجود نماد متناظر، `symbol_id` را نیز نگه می‌دارد. این زنجیره اجازه می‌دهد نتیجه بازیابی با شناسه قطعه آغاز و سپس با مسیر فایل، مشخصات نماد و محدوده خطوط قطعی تکمیل شود. در نتیجه فراداده قابل اعتماد از یک رابطه پایگاه داده به دست می‌آید، نه از متن آزاد ذخیره‌شده کنار بردار.

۳-۹- طراحی لایه ذخیره‌سازی

۳-۹-۱- `SQLite` به عنوان منبع اصلی حقیقت

`SQLite` وضعیت پروژه‌ها، فایل‌های منبع، نمادها، قطعه‌ها و کارهای پردازشی نمایه‌سازی را نگه می‌دارد. جدول پروژه علاوه بر مسیر ریشه، نسخه طرح نمایه و `vector_generation` فعال را ثبت می‌کند. محدودیت‌های یکتا برای مسیر فایل، هویت نماد و `chunk_id` از ایجاد رکوردهای متناقض جلوگیری می‌کنند. جایگزینی کامل

^۱ Snapshot

یا اعمال تغییر افزایشی در تراکنش انجام می‌شود تا مجموعه فراداده به‌صورت نیمه‌کاره قابل مشاهده نباشد.

۳-۹-۲ ChromaDB به‌عنوان نمایه برداری قابل جایگزینی

ChromaDB نگاشت chunk_id به بردار تعبیه‌سازی را برای جست‌وجوی شباهت نگه می‌دارد. این لایه به‌عمد منبع حقیقت نیست و می‌تواند از روی SQLite و ارائه‌دهنده مشخص دوباره ساخته شود. پیش از استفاده، هویت ارائه‌دهنده و مدل، نقطه‌ی پایانی^۱ بدون اطلاعات حساس، ابعاد بردار، نسخه طرح و مجموعه دقیق شناسه‌ها کنترل می‌شود. جدول ۳-۳ تقسیم مسئولیت دو لایه را خلاصه می‌کند.

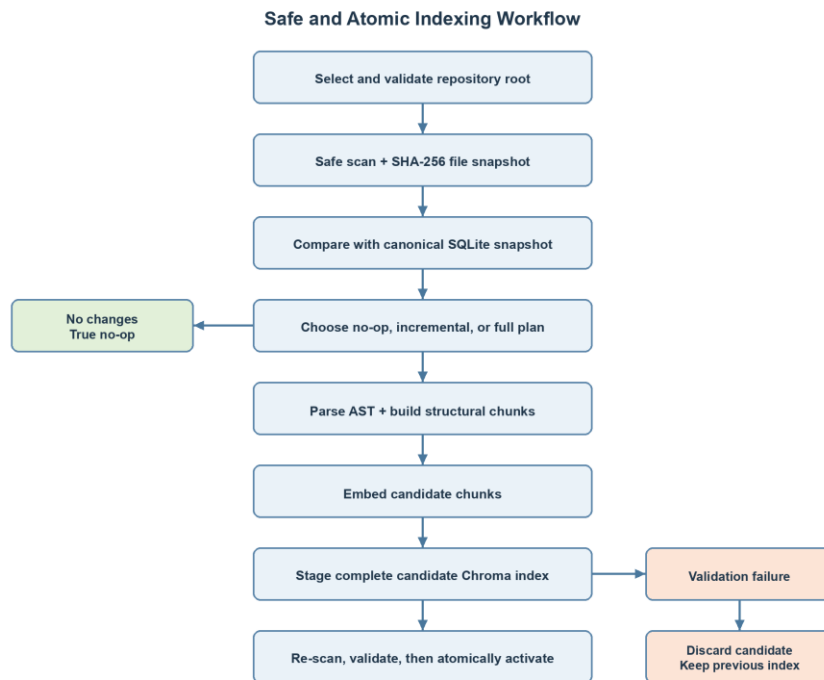
جدول ۳-۳: تقسیم مسئولیت ChromaDB و SQLite

موضوع	SQLite	ChromaDB
هویت پروژه و فایل	مرجع قطعی	نگهداری نمی‌شود
نماد و محدوده خطوط	مرجع قطعی	مرجع قابل اعتماد نیست
متن و هش قطعه	مرجع قطعی	برای کنترل اتصال استفاده می‌شود
بردار تعبیه‌سازی	هویت و نسل نمایه	ذخیره و جست‌وجوی بردار
بازیابی پس از خرابی	مبنای بازسازی	نمایه قابل جایگزینی

۳-۱۰-۱ فرایند نمایه‌سازی

نمایه‌سازی به‌صورت ساخت نمایه‌ی نامزد و فعال‌سازی کنترل‌شده طراحی شده است. شکل ۳-۲ مراحل اصلی و شاخه شکست را نشان می‌دهد. اصل کلیدی آن است که هیچ مجموعه برداری ناقص یا ناسازگار جای نمایه فعال را نگیرد.

¹ Endpoint



شکل ۳-۲: جریان نمایه‌سازی امن و فعال‌سازی اتمی

۳-۱۰-۱- نمایه‌سازی اولیه

در نخستین اجرا همه فایل‌های پذیرفته‌شده تحلیل و قطعه‌بندی می‌شوند. سرویس پیش از نوشتن نهایی، دسترسی ارائه‌دهنده و ابعاد خروجی تعبیه‌سازی را پیش‌آزمایی می‌کند، بردار همه قطعه‌ها را می‌سازد و یک مجموعه Chroma مدیریت‌شده و موقت ایجاد می‌نماید. سپس پروژه در SQLite رزرو و نمایه‌ی نامزد برای فعال‌سازی آماده می‌شود.

۳-۱۰-۲- نمایه‌سازی افزایشی

در اجراهای بعدی تصویر لحظه‌ای جدید با رکوردهای SQLite مقایسه می‌شود. اگر هیچ تغییری وجود نداشته باشد فرایند واقعاً بدون عملیات است. در غیر این صورت تنها فایل‌های افزوده یا تغییر کرده تحلیل نحوی و بردارسازی می‌شوند، رکورد فایل‌های حذف‌شده کنار گذاشته می‌شود و بردار قطعه‌های بدون تغییر پس از کنترل `content_hash`، `project_id`، مدل و ابعاد دوباره استفاده می‌گردد. این روش هزینه را کاهش می‌دهد بی‌آنکه صحت مجموعه نهایی قربانی شود.

۳-۱۰-۳- ساخت نمایه‌ی نامزد

صرف‌نظر از کامل یا افزایشی بودن، خروجی مورد انتظار یک نمایه‌ی نامزد کامل است؛ یعنی باید تمام `chunk_id`های نسل جدید را در بر داشته باشد. مجموعه جدید با نام مدیریت‌شده مرحله‌بندی می‌شود و تا

پیش از فعال سازی از دید بازیابی عادی جدا می ماند. نمایه فعال قبلی در این مرحله حذف یا بازنویسی نمی شود.

۳-۱۰-۴- اعتبارسنجی پیش از فعال سازی

پیش از ثبت نهایی، مخزن دوباره پویش می شود تا تغییر همزمان فایل ها در حین نمایه سازی تشخیص داده شود. مجموعه شناسه های واقعی و مورد انتظار باید دقیقاً برابر باشند؛ ارائه دهنده، مدل، هش نقطه ی پایانی، ابعاد، نسخه طرح و نسل بردار نیز کنترل می شوند. فقط پس از موفقیت این آزمون ها اشاره گر مجموعه فعال در همان مرز تراکنشی که فراداده SQLite را به روزرسانی می کند تغییر می یابد.

۳-۱۰-۵- حفظ نمایه قبلی در صورت شکست

اگر تحلیل نحوی، تعبیه سازی، نوشتن بردار، بازپویش یا اعتبارسنجی شکست بخورد، نمایه ی نامزد دور ریخته می شود. اگر شکست هنگام فعال سازی رخ دهد، تابع بازخوانی^۱ وضعیت برداری را به مجموعه پیشین برمی گرداند و تراکنش SQLite نیز بازگردانی^۲ می شود. بنابراین شکست کنترل شده به معنی از دست رفتن نمایه کامل قبلی یا مشاهده مخلوط دو نسل نیست.

۳-۱۱- روش های بازیابی

۳-۱۱-۱- بازیابی واژگانی

بازیابی واژگانی پرسش و فیلدهای قطعه را به توکن های Unicode تبدیل می کند و هم پوشانی دقیق را می سنجد. نام واجد شرایط با وزن ۳، مسیر منبع با وزن ۲ و کد و embedding_text هر کدام با وزن ۱ در امتیاز مشارکت دارند. مرتب سازی در تساوی با مسیر، خط شروع و chunk_id قطعی می شود. این روش برای نام های دقیق مانند create_project یا validate_credentials سریع و قابل تفسیر است و به ارائه دهنده برداری نیاز ندارد.

۳-۱۱-۲- بازیابی معنایی

در روش معنایی، پرسش با ارائه دهنده و مدل سازگار با نمایه فعال به بردار تبدیل می شود. پیش از پرس و جو^۳ هویت و ابعاد تعبیه سازی کنترل می گردند. نتیجه Chroma با project_id محدود و سپس هر نتیجه^۴ با chunk_id با فراداده SQLite تکمیل می شود؛ نبودن رکورد متناظر خطای سازگاری است و با حدس زدن فراداده جبران نمی شود. این روش برای پرسش هایی که واژگانشان با نام کد متفاوت است ارتباط مفهومی را آشکار می کند.

^۱ Callback

^۲ Rollback

^۳ Query

^۴ Hit

در پیاده‌سازی CodeCompass، شباهت کسینوسی برای رتبه‌بندی برداری انتخاب شد؛ زیرا هدف، مقایسه جهت و الگوی مؤلفه‌های بردار پرسش و قطعه کد است و بزرگی مطلق بردار نباید به‌تنهایی رتبه را تعیین کند. این انتخاب با نمایه ChromaDB که برای فاصله کسینوسی پیکربندی شده و با خروجی مدل‌های تعبیه‌سازی مورد استفاده سامانه سازگار است. معیارهایی مانند فاصله اقلیدسی و ضرب داخلی نیز امکان‌پذیرند، اما در این پروژه آزمایش کنترل‌شده‌ای برای مقایسه این معیارها انجام نشده است؛ از این رو کسینوسی به‌عنوان تصمیم پیاده‌سازی سازگار با معماری موجود انتخاب شده و ادعای برتری عمومی آن مطرح نمی‌شود.

۳-۱۱-۳- بازیابی ترکیبی

بازیابی ترکیبی نتایج واژگانی و معنایی را با RRF ادغام می‌کند. برای هر قطعه امتیاز $1/(60+\text{rank})$ از هر فهرستی که در آن حضور دارد جمع می‌شود. استفاده از رتبه به‌جای مقیاس خام، نیاز به هم‌مقیاس کردن امتیاز واژگانی و شباهت برداری را حذف می‌کند. قطعه تکراری با `chunk_id` ادغام و مرتب‌سازی نهایی به‌صورت قطعی انجام می‌شود.

۳-۱۱-۴- محدودسازی و رتبه‌بندی نتایج

پارامتر `limit` باید مثبت باشد و تنها بالاترین نتایج پس از رتبه‌بندی بازگردانده می‌شوند. در پرسش و پاسخ، اگر نام دقیق نماد در پرسش آمده باشد سرویس قطعه‌های همان نام واجد شرایط یا نام کوتاه را در اولویت قرار می‌دهد. این تمرکز از غالب‌شدن قطعه‌های عمومی‌تر بر هدف صریح کاربر جلوگیری می‌کند.

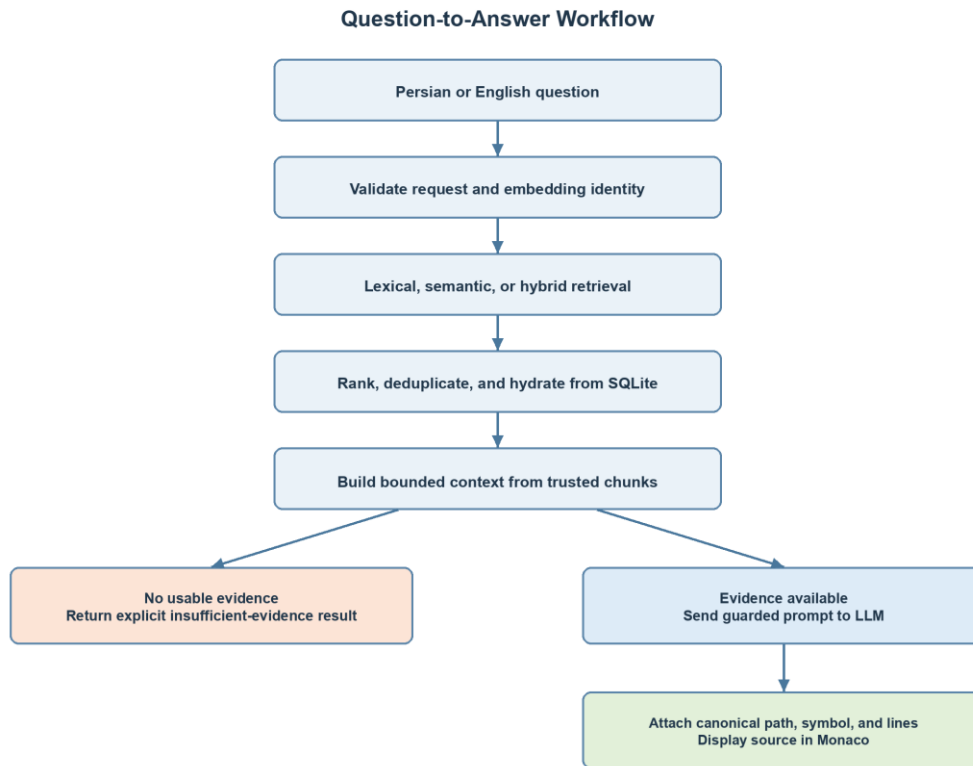
۳-۱۲- ساخت زمینه برای RAG

سازنده زمینه نتایج را با `chunk_id` تکراری‌زدایی و با ترتیب قطعی پردازش می‌کند. هر قطعه همراه با کد و برجسب منبع وارد زمینه می‌شود، اما فقط اگر افزودن کامل آن از `max_context_chars` عبور نکند. قطعه‌ها نصفه بریده نمی‌شوند و تعداد موارد حذف‌شده نیز ثبت می‌شود. در درخواست پیش‌فرض، پنج نتیجه بازیابی و بودجه ۶۰۰۰ کاراکتری به کار می‌رود؛ این مقادیر از طریق API قابل تنظیم‌اند.

محدودکردن زمینه دو اثر دارد: ورودی مدل کوچک و قابل پیش‌بینی باقی می‌ماند و احتمال آمیختن بخش‌های نامرتب کاهش می‌یابد. فراداده ارجاع هم‌زمان در ساختار داخلی نگه داشته می‌شود، اما وظیفه مدل فقط نوشتن متن پاسخ است. بدین ترتیب رشته‌ای که برای مدل ارسال می‌شود با رکوردهای قطعی مورد استفاده برای ارجاع اشتباه گرفته نمی‌شود.

۳-۱۳- تولید پاسخ مستند

شکل ۳-۳ جریان پرسش تا پاسخ را نشان می‌دهد. مسیر شکست شواهد عمداً پیش از فراخوانی مدل جدا شده است تا سامانه در نبود زمینه کافی پاسخ ظاهراً قانع‌کننده اما بی‌پشتوانه تولید نکند.



شکل ۳-۳: جریان پرسش تا پاسخ مستند و نمایش منبع

۳-۱۳-۱- پاسخ مبتنی بر شواهد

دستور مدل سامانه به LLM می‌گوید فقط از زمینه استفاده کند، در برابر دستورهای موجود در کد و متن بازیابی شده بی‌اعتنا باشد و به زبان پرسش پاسخ دهد. دمای صفر و بودجه محدود توکن برای کاهش نوسان و زیاده‌گویی به کار می‌روند. خروجی مدل به عنوان توضیح طبیعی پذیرفته می‌شود، نه به عنوان منبع هویت کد.

۳-۱۳-۲- رفتار شواهد ناکافی

اگر هیچ قطعه قابل استفاده در زمینه وجود نداشته باشد، سرویس اصلاً LLM را فراخوانی نمی‌کند و نتیجه صریح کافی نبودن شواهد را با فهرست ارجاع خالی بازمی‌گرداند. خطای ناسازگاری تعبیه‌سازی یا شکست ارائه‌دهنده نیز به پیام کنترل شده تبدیل می‌شود و نباید به پاسخ حدسی یا امتیاز ساختگی منجر گردد.

۳-۱۳-۳- جداسازی متن مدل از اطلاعات قطعی

مسئولیت‌ها در مرز روشنی تقسیم شده‌اند. مدل می‌تواند خلاصه، توضیح رفتار و بیان طبیعی تولید کند؛ در مقابل سامانه مالک `project_id`، مسیر فایل، نام نماد، محدوده خطوط، `chunk_id`، `hash`، حقایق تابع و ارجاع است. این جداسازی مهم‌ترین سازوکار اعتماد CodeCompass است و امکان ممیزی هر پاسخ را فراهم می‌سازد.

۳-۱۴- تولید ارجاع قابل اعتماد

۳-۱۴-۱- مسیر فایل

مسیر نمایش داده شده از `relative_path` ثبت شده هنگام پوشش گرفته می شود. این مسیر نسبت به ریشه پروژه است، قابل حمل باقی می ماند و مسیر مطلق خصوصی رایانه را وارد خروجی عمومی نمی کند. محتوا نیز فقط از فایل متعلق به پروژه منتخب خوانده می شود.

۳-۱۴-۲- نام نماد

نام نماد از رکورد تحلیل گر نحوی و رابطه قطعه با نماد استخراج می شود `qualified_name`. برای رفع ابهام دامنه به کار می رود. در مستندسازی، انتخاب با `symbol_id`, `chunk_id` پایدار یا نام دقیق ممکن است؛ اگر نام کوتاه چند تطابق داشته باشد سرویس فهرست مرتب گزینه ها را برمی گرداند و به صورت خاموش یکی را انتخاب نمی کند.

۳-۱۴-۳- شماره خطوط

`start_line` و `end_line` از AST و رکورد قطعه به دست می آیند و هنگام تکمیل فراداده ی نتیجه بازایی دوباره از SQLite خوانده می شوند API. برای نمایش محتوا همین محدوده را تحویل رابط می دهد و Monaco خطوط متناظر را آشکار و برجسته می کند. در نتیجه شماره خط بخشی از قرارداد داده است، نه تزئین متن پاسخ.

۳-۱۴-۴- جلوگیری از جعل ارجاع توسط مدل

متن تولید شده برای استخراج مسیر، نام نماد یا خط تجزیه نمی شود. ارجاع ها پس از پاسخ از زمینه داخلی ساخته و به خروجی متصل می شوند. در سرویس مستندسازی نیز هر ارجاع بازگشتی با رکورد مرجع قطعی جایگزین می گردد. بنابراین نوشتن یک مسیر یا شماره خط خیالی توسط مدل نمی تواند آن را به ارجاع رسمی رابط تبدیل کند.

۳-۱۵- تولید مستندات تابع

۳-۱۵-۱- استخراج قطعی اطلاعات تابع

سرویس Documentation ابتدا تابع یا متد را بدون ابهام تعیین و متن قطعه را با AST دوباره تحلیل می کند. پارامترها همراه با نوع، مقدار پیش فرض و `kind`، اعلان نوع بازگشتی، `async` بودن، `raise` های صریح، فراخوانی های مستقیم و وجود `return` صریح استخراج می شوند. تعریف های تودرتو، کلاس های داخلی و Lambda در فهرست فراخوانی مستقیم دخالت داده نمی شوند. فراداده تحلیل گر نحوی نیز با واقعیت متن قطعه تطبیق داده می شود تا ناسازگاری پنهان نماند.

۳-۱۵-۲- تولید توضیح توسط مدل

حقایق قطعی همراه با کد به LLM داده می‌شوند و از مدل یک JSON^۱ محدود شامل summary, behavior, parameter_descriptions, return_description و notes خواسته می‌شود. تولید در دمای صفر انجام می‌گیرد و برای خروجی فارسی، در صورت رعایت‌نشدن زبان تنها یک تلاش مجدد محدود مجاز است. مستندات برحسب درخواست ساخته می‌شوند و در پایگاه داده ذخیره یا در حافظه‌ی نهان نگهداری نمی‌شوند.

۳-۱۵-۳- اعتبارسنجی ساختار خروجی

خروجی مدل باید JSON معتبر و مطابق طرح‌واره^۲ باشد؛ نام پارامترهای توضیح‌داده‌شده باید دقیقاً با مجموعه پارامترهای قطعی برابر باشد. فیلد اضافی، پارامتر حذف‌شده یا هویت منبع تولیدشده توسط مدل پذیرفته نمی‌شود. نتیجه نهایی از ترکیب حقایق سیستمی و نثر مدل ساخته و با ارجاع قطعی همان تابع ارائه می‌گردد.

۳-۱۶- معماری ارائه‌دهنده‌ها

تعبیه‌سازی و LLM پشت قراردادهای جداگانه ارائه‌دهنده قرار دارند. تنظیمات هرکدام شامل نوع ارائه‌دهنده، مدل، نشانی پایه^۳ و در صورت نیاز کلید^۴ API است و می‌توان آن‌ها را مستقل انتخاب کرد. این طراحی امکان استفاده از یک مدل محلی برای تولید و یک نقطه‌ی پایانی سازگار دیگر برای تعبیه‌سازی، یا برعکس، را فراهم می‌کند.

۳-۱۶-۱- اولاما (Ollama)

ارائه‌دهنده محلی Ollama از /api/embed برای بردارسازی و /api/generate برای تولید استفاده می‌کند. این گزینه امکان اجرای کامل جریان روی رایانه کاربر را فراهم می‌سازد، مشروط به نصب مدل‌های لازم. پاسخ‌های شبکه، خالی بودن بردار و شکل نامعتبر داده پیش از استفاده کنترل می‌شوند.

۳-۱۶-۲- ارائه‌دهندگان سازگار با OpenAI

ارائه‌دهنده نوع openai_compatible از نقطه‌های پایانی /embeddings و /chat/completions استفاده می‌کند و برای سرویس‌های دارای API مشابه قابل تنظیم است. کلید در نمایش شیء پنهان می‌شود، در SQLite یا خروجی ثبت‌شده^۵ پروژه ذخیره نمی‌گردد و خطای خام حاوی جزئیات حساس مستقیماً به کاربر برگردانده نمی‌شود.

^۱ JavaScript Object Notation (JSON)

^۲ Schema

^۳ Base URL (Uniform Resource Locator)

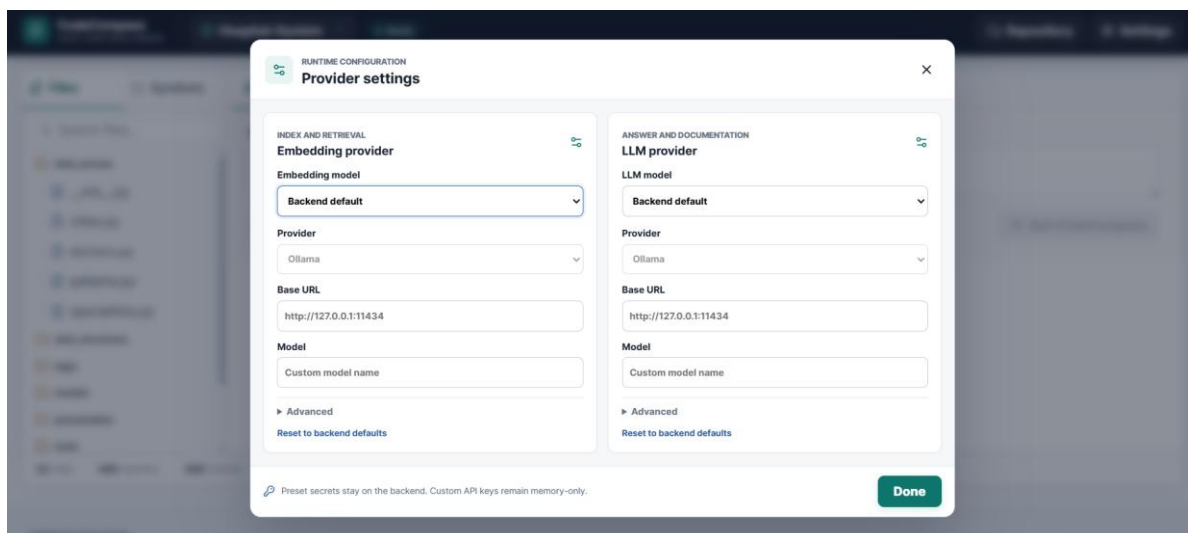
^۴ API key

^۵ Artifact

۳-۱۶-۳- استقلال مدل تعبیه‌سازی و LLM

مدل تعبیه‌سازی بر قابلیت جست‌وجوی معنایی و سازگاری نمایه اثر دارد، درحالی‌که LLM فقط متن پاسخ یا توضیح را تولید می‌کند. تغییر LLM نیازمند بازنمایه‌سازی نیست؛ اما تغییر ارائه‌دهنده یا مدل تعبیه‌سازی، نقطه‌ی پایانی مؤثر یا ابعاد بردار مستلزم ساخت نمایه سازگار جدید است. بازیابی واژگانی حتی در نبود ارائه‌دهنده معنایی همچنان قابل استفاده باقی می‌ماند.

تنظیم مستقل مدل تعبیه‌سازی و مدل زبانی در رابط کاربری نیز در شکل ۳-۴ نشان داده شده است. این پنجره امکان انتخاب ارائه‌دهنده، نشانی پایه و مدل هر بخش را بدون آمیختن مسئولیت بازیابی و تولید فراهم می‌کند.



شکل ۳-۴: تنظیم مستقل ارائه‌دهنده تعبیه‌سازی و مدل زبانی

۳-۱۷-۱- رابط برنامه‌نویسی FastAPI

FastAPI یک API محلی و صریح برای رابط کاربری فراهم می‌کند. نقطه‌های پایانی نمایه‌سازی هم اجرای همگام و هم کار پردازشی پس‌زمینه با پایش دوره‌ای وضعیت را پشتیبانی می‌کنند. گروه‌های اصلی API در جدول ۳-۴ آمده‌اند. ورودی‌ها با مدل‌های داده اعتبارسنجی و خطاهای دامنه به پاسخ ¹HTTP کنترل‌شده تبدیل می‌شوند.

جدول ۳-۴: گروه‌های اصلی API سامانه

گروه	عملیات اصلی
سلامت و پروژه	Health، فهرست پروژه‌ها و جزئیات پروژه

¹ Hypertext Transfer Protocol (HTTP)

گروه	عملیات اصلی
نمایه‌سازی	ایجاد نمایه همگام، آغاز کار پردازشی و مشاهده وضعیت کار پردازشی
منبع	فهرست فایل‌ها، محتوای فایل و فهرست نمادها
بازیابی	Search واژگانی، معنایی یا ترکیبی
پرسش و پاسخ	Ask با زمینه محدود و ارجاع قطعی
مستندسازی	Documentation برای تابع یا متد منتخب
ارزیابی	خلاصه، Performance و نتایج نهایی پایان‌نامه

API مسیر بارگذاری مخزن، احراز هویت، نشست^۱ چندکاربره، صف توزیع‌شده، لغو کار پردازشی، ارسال جریانی پاسخ یا مدیریت CORS^۲ برای استقرار ابری ارائه نمی‌دهد؛ زیرا این موارد خارج از محدوده سامانه محلی پایان‌نامه‌اند. نقطه‌های پایانی ارزیابی نیز داده‌های منجمد را می‌خوانند و آزمایش جدید را هنگام مشاهده داشبورد اجرا نمی‌کنند.

۳-۱۸- رابط کاربری React و Vite

رابط وب با React 19 و Vite پیاده‌سازی شده است و یک فضای کاری واحد برای انتخاب پروژه، مرور ساختار، جست‌وجو، پرسش و پاسخ، مستندسازی و ارزیابی فراهم می‌کند. متن فارسی با جهت مناسب نمایش داده می‌شود و تنظیمات تعبیه‌سازی و LLM به‌صورت مستقل در رابط قابل انتخاب‌اند.

۳-۱۸-۱- مدیریت پروژه

کاربر پروژه فعال را از انتخابگر پروژه^۳ انتخاب می‌کند یا مسیر مخزن را برای نمایه‌سازی وارد می‌نماید. رابط وضعیت کار پردازشی را به‌صورت دوره‌ای بررسی می‌کند و مرحله جاری، نتیجه و خطا را نمایش می‌دهد. پس از اتمام، فهرست فایل‌ها و نمادها با پروژه فعال همگام می‌شود.

۳-۱۸-۲- مرور فایل‌ها و نمادها

کاوشگر پروژه^۴ دو نمای فایل و نماد، فیلتر و ساختار درختی ارائه می‌دهد. انتخاب یک مورد، محتوای منبع یا موقعیت نماد را بدون خروج از فضای کاری در دسترس قرار می‌دهد. این قابلیت برای مطالعه سلسله‌مراتب مخزن مستقل از پرسش مدل مفید است.

¹ Session

² Cross-Origin Resource Sharing (CORS)

³ Project Switcher

⁴ Project Explorer

۳-۱۸-۳ جست و جو (Search)

زبان Search امکان انتخاب روش semantic, lexical یا hybrid و تعیین limit را فراهم می‌کند. هر نتیجه امتیاز، نوع قطعه، نام نماد، مسیر و محدوده خطوط را نشان می‌دهد و می‌توان مستقیماً کد همان نتیجه را باز کرد.

۳-۱۸-۴ پرسش و پاسخ (Ask)

در زبان Ask کاربر پرسش فارسی یا انگلیسی، روش بازیابی، تعداد شواهد، بودجه زمینه و محدودیت توکن پاسخ را تعیین می‌کند. پاسخ در کنار ارجاع‌ها نمایش داده می‌شود. اگر شواهد کافی نباشد همان وضعیت صریح بخش پشتیبان به کاربر منتقل خواهد شد.

۳-۱۸-۵ مستندسازی (Documentation)

زبان Documentation شناسه یا نام تابع/متد و زبان خروجی را دریافت می‌کند. در ابهام نام، گزینه‌های کاندیدا نمایش داده می‌شوند. خروجی، حقایق امضا و رفتار استخراج‌شده را از توضیح تولیدی جدا و محل منبع را نیز قابل بازکردن ارائه می‌کند.

۳-۱۸-۶ نمایش خطوط مورد استناد با Monaco

Code Drawer از Monaco Editor استفاده می‌کند. هنگام انتخاب ارجاع، فایل مناسب بارگیری، خط هدف آشکار و بازه ارجاع برجسته می‌شود. این رفتار به کاربر اجازه می‌دهد صحت پاسخ را با متن واقعی مخزن مقایسه کند و به اعتماد صرف به نثر مدل نیاز نداشته باشد.

۳-۱۸-۷ داشبورد ارزیابی

پنل ارزیابی^۱ نتایج ارزیابی رسمی و نهایی پایان‌نامه را از خروجی‌های ثبت‌شده منجمد نشان می‌دهد. معیارها، دیدگاه‌های مختلف، جزئیات موارد آزمون و خروجی‌های قابل انتشار تفکیک شده‌اند. داشبورد مقدار اندازه‌گیری‌شده، مقدار در دسترس نبود و اجرای شکست‌خورده را یکی فرض نمی‌کند و داده خام آزمایش را بازنویسی نمی‌نماید.

۳-۱۹- ملاحظات امنیت، حریم خصوصی و مدیریت خطا

امنیت از محدودسازی ورودی آغاز می‌شود: مسیرهای بیرون از ریشه، پیوندهای نمادین، فایل‌های غیرپایتون، اسرار env^۲، VCS^۲، محیط مجازی، کش و خروجی ساخت نمایه نمی‌شوند. متن کد بازیابی‌شده نیز

^۱ Evaluation Panel

^۲ Version Control System (VCS)

در دستور مدل به عنوان داده غیر قابل اعتماد معرفی می شود تا دستور احتمالی داخل کد نقش دستور سیستمی^۱ پیدا نکند.

از نظر حریم خصوصی، Ollama امکان اجرای محلی را می دهد. در صورت انتخاب ارائه دهنده سازگار با OpenAI، کاربر آگاهانه نقطه پایانی بیرونی را پیکربندی می کند و قطعه های لازم ممکن است برای آن سرویس ارسال شوند؛ بنابراین محلی محور بودن به معنی ممنوعیت ارائه دهنده شبکه نیست. کلید API در تنظیمات پایدار پروژه و خروجی های ثبت شده ارزیابی ذخیره نمی شود و هش نقطه پایانی نیز بدون اطلاعات احراز هویت و پرس و جو حساس ساخته می گردد.

خطاهای مسیر، خواندن فایل، SyntaxError، ورودی نامعتبر، نبود نماد، ابهام نماد، ناسازگاری مدل یا ابعاد تعبیه سازی، پاسخ نامعتبر ارائه دهنده و شکست نمایه نامزد به وضعیت های مشخص تبدیل می شوند. اصل بازیابی پذیری این است که شکست یک عملیات جدید نباید داده معتبر قبلی را تخریب کند. همچنین در ارزیابی، خروجی در دسترس نبودن یا شکست به صفر تبدیل نمی شود؛ زیرا صفر یک اندازه گیری واقعی است و معنای متفاوتی دارد.

۳-۲۰- جمع بندی

در این فصل طراحی و پیاده سازی نهایی CodeCompass از انتخاب مخزن تا مشاهده کد مورد استناد تشریح شد. پویش امن و AST، ساختار قطعی مخزن را استخراج می کنند؛ قطعه بندی ساختار آگاه و شناسه ها و هش های پایدار، اتصال داده ها را می سازند؛ SQLite منبع حقیقت و ChromaDB نمایه برداری قابل جایگزینی است؛ و فرایند نمایه نامزد از فعال شدن داده ناقص جلوگیری می کند.

در مسیر پرسش، سه روش واژگانی، معنایی و ترکیبی شواهد را انتخاب می کنند، زمینه محدود در اختیار LLM قرار می گیرد و در نبود شواهد کافی پاسخ حدسی تولید نمی شود. مهم تر از همه، مسیر فایل، نام نماد و خطوط از فراداده سامانه ساخته می شوند و از متن مدل مستقل اند. FastAPI و رابط React/Vite این قابلیت ها را در قالب Search، Ask، Documentation، مرور منبع و داشبورد ارزیابی یکپارچه می کنند. فصل بعد روش آزمایش، داده ها، معیارها و نتایج ارزیابی نسخه نهایی سامانه را ارائه خواهد کرد.

¹ System Instruction

فصل چهارم

ارزیابی و نتایج پیاده‌سازی سامانه CODECOMPASS

۴-۱- مقدمه

در فصل پیش، معماری، اجزای اصلی و فرایند پیاده‌سازی سامانه CodeCompass تشریح شد. در این فصل، عملکرد نسخه نهایی سامانه از جنبه‌های نمایه‌سازی، بازیابی، تولید پاسخ، مستندسازی و قابلیت اعتماد ارجاع‌ها ارزیابی می‌شود. هدف آن است که مشخص شود آیا سامانه می‌تواند مخازن پایتون را به صورت قابل اعتماد تحلیل کند، پرسش‌های فارسی و انگلیسی را به شواهد مرتبط پیوند دهد و پاسخ‌هایی تولید کند که محل استناد آن‌ها در کد منبع قابل بررسی باشد.

ارزیابی هر لایه به صورت جداگانه انجام شد؛ زیرا موفقیت بازیابی الزاماً به معنی تولید پاسخ مناسب نیست و روان بودن پاسخ نیز صحت منبع آن را تضمین نمی‌کند. در ادامه، ابتدا طراحی آزمایش و نتایج ارزیابی نهایی سامانه ارائه می‌شود و سپس با یک آزمایش حذف مؤلفه‌ها^۱، راهبردهای ارسال کل مخزن، بازیابی واژگانی، بازیابی معنایی، بازیابی ترکیبی و عامل ابزارمحور OpenCode مقایسه می‌شوند.

۴-۲- اهداف و پرسش‌های ارزیابی

ارزیابی نهایی برای پاسخ‌گویی به این پرسش‌ها طراحی شد: آیا نمایه برداری با داده‌های قطعی SQLite کامل و سازگار است؛ روش‌های واژگانی، معنایی و ترکیبی در بازیابی شواهد چه عملکردی دارند؛ مدل تعبیه‌سازی و مدل زبانی چه اثری بر نتیجه دارند؛ پاسخ‌ها و مستندات تا چه اندازه صحیح و متکی بر شواهد هستند؛ و ارجاع‌های مسیر، نماد و خطوط تا چه حد با کد واقعی مطابقت دارند.

آزمایش مقایسه راهبرد زمینه سه پرسش تکمیلی را بررسی کرد: آیا انتخاب زمینه با RAG از ارسال کل مخزن به GLM^۲ بهتر است؛ آیا تعبیه‌سازی نسبت به بازیابی صرفاً واژگانی سود قابل اندازه‌گیری دارد؛ و آیا عامل OpenCode که با ابزارهای خواندن و جست‌وجو به یک نسخه مشخص از مخزن دسترسی دارد، پاسخ‌هایی

^۱ Ablation Study

^۲ General Language Model (GLM)

با کیفیت بالاتر از RAG معنایی تولید می‌کند.

۳-۴- طراحی ارزیابی

۳-۴-۱- مخازن مورد ارزیابی

ارزیابی نهایی روی سه مخزن پایتون با اندازه و ساختار متفاوت انجام شد. هر مخزن روی یک نسخه مشخص ثابت شد تا تغییرات بعدی کد بر نتایج اثر نگذارد. Hospital-System منطق‌های کاربری و ساختارهای داده، CS-Bookstore یک برنامه وب با جریان‌های اعتبارسنجی و ثبت داده، و CodeCompass اجزای نمایه‌سازی، بازیابی، ارائه‌دهندگان و رابط برنامه‌نویسی را پوشش می‌دهد. مشخصات مخازن مورد ارزیابی در جدول ۴-۱ ارائه شده است.

جدول ۴-۱: مشخصات مخازن ارزیابی نهایی

مخزن	فایل‌های پایتون	نمادها	قطعه‌ها
Hospital-System	۳۳	۴۰۹	۴۰۹
CS-Bookstore	۵۶	۱۴۴	۱۴۴
CodeCompass	۱۰۰	۱۲۳۶	۱۲۳۶

۳-۴-۲- مجموعه آزمون و اندازه آزمایش

مجموعه آزمون شامل پرسش‌های فارسی و انگلیسی با دشواری‌های آسان، متوسط و دشوار بود. پرسش‌ها، پاسخ‌های مرجع، نسخه مخازن و تنظیمات پیش از اجرای نهایی ثابت شدند و هنگام مقایسه مدل‌ها تغییر نکردند. اندازه مجموعه آزمون و تعداد اجراها در جدول ۴-۲ خلاصه شده است. افزون بر این، یک مجموعه ارزیابی^۱ رسمی مستقل شامل ۶۰ پرسش و ۳۰ مفهوم دوزبانه برای ارزیابی بازیابی در شرایط ثابت استفاده شد که نتایج آن در بخش ۴-۶-۴ گزارش می‌شود.

^۱ Benchmark

جدول ۴-۲: اندازه مجموعه ارزیابی نهایی

وظیفه	واحدهای ثابت شده	تعداد اجرا
جست و جوی کد	۱۸ مفهوم دوزبانه و ۳۶ پرسش	۳۲۴
پرسش و پاسخ	۱۲ پرسش	۷۲
مستندسازی تابع	۹ نماد فارسی	۱۸

۴-۳-۳- مدل ها و تنظیمات

سه بازوی تعبیه سازی شامل مدل محلی latest:nomic-embed-text-local از طریق Ollama، مدل Gemini Embedding001 و مدل Gemini Embedding2 بودند. مدل محلی بردارهای ۷۶۸ بعدی و دو مدل Gemini بردارهای ۳۰۷۲ بعدی تولید می کردند. هویت ارائه دهنده، نشانی مؤثر سرویس، نام مدل و ابعاد بردار در فراداده نمایه ثبت و پیش از بازیابی معنایی اعتبارسنجی شدند.

برای تولید پاسخ، مدل محلی Qwen و مدل Flash 5.3 GLM مقایسه شدند. در پرسش و پاسخ، دمای تولید صفر، سقف خروجی ۱۲۰۰ توکن، روش بازیابی ترکیبی و سقف زمینه ۶۰۰۰ نویسه بود. در مستندسازی، سقف خروجی ۲۴۰۰ توکن تعیین شد. هنگام مقایسه مدل های زبانی، پرسش، شواهد، دستور مدل و تنظیمات تولید ثابت باقی ماندند.

۴-۳-۴- معیارهای ارزیابی

برای بازیابی از نرخ موفقیت در رتبه اول و سوم^۱، موفقیت تا رتبه پنجم و دهم و میانگین معکوس رتبه تا ده نتیجه^۲ استفاده شد. این معیارها به ترتیب حضور نتیجه صحیح در رتبه های نخست و جایگاه آن در فهرست را اندازه گیری می کنند. بازخوانی شواهد نیز میزان حضور حقایق مرجع در زمینه ساخته شده برای مدل را نشان می دهد.

پاسخ های تولید شده از نظر پوشش حقایق، صحت، اتکاپذیری به شواهد، کامل بودن، خوانایی فارسی و مفید بودن ارزیابی شدند. اجراهای ناموفق یا خروجی های ناموجود نمره صفر نگرفتند و جدا از میانگین کیفیت گزارش شدند؛ زیرا شکست ارائه دهنده با کیفیت محتوایی مدل یکسان نیست.

۴-۴- نتایج عملکرد کارکردی سامانه

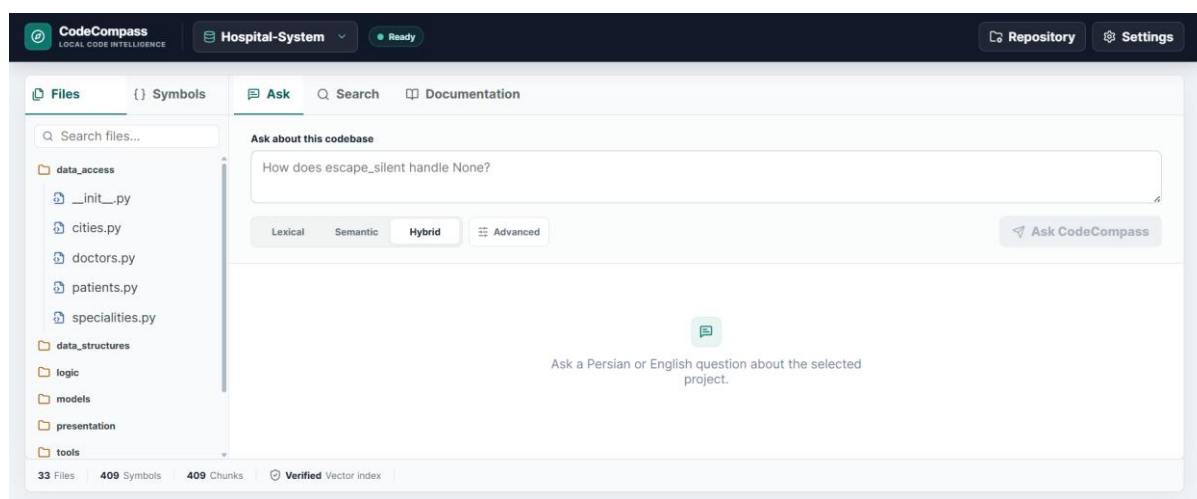
نسخه نهایی CodeCompass امکان انتخاب و نمایه سازی مخزن، مرور فایل ها و نمادها، جست و جوی

¹ Hit at k (Hit@k)

² Mean Reciprocal Rank at 10 (MRR@10)

واژگانی، معنایی و ترکیبی، طرح پرسش، تولید مستندات تابع و مشاهده خطوط مورد استناد را فراهم می‌کند. نتایج جست‌وجو همراه با مسیر، نماد و محدوده خطوط نمایش داده می‌شوند و انتخاب ارجاع، کد متناظر را در Monaco آشکار و برجسته می‌کند.

در بخش Ask، پاسخ طبیعی در کنار ارجاع‌های قطعی ارائه می‌شود و در نبود شواهد کافی، سامانه وضعیت شواهد ناکافی را اعلام می‌کند. در بخش Documentation نیز حقایق استخراج‌شده از کد از نثر مدل جدا هستند. شکل ۴-۱ فضای کاری اصلی و شکل ۴-۲ داشبورد نتایج منجمد ارزیابی را نشان می‌دهند.



شکل ۴-۱: فضای کاری اصلی و انتخاب روش ارزیابی در CodeCompass

SCIENTIFIC EVALUATION

Evaluation and performance

Official retrieval

Final thesis evaluation

Context strategy experiment

Frozen evaluation

Final thesis evaluation

2 repositories — 35 search queries — 80 human-review records

Overview

Search

QA

Documentation

Reliability

RESEARCH QUESTION

How do three embedding models and two LLMs affect retrieval, grounded QA, Persian readability, documentation, and execution reliability under frozen conditions?

CONCLUSION

There was no universal model winner: Gemini Embedding 2 led pure semantic retrieval, Gemini Embedding 001 produced the highest mean downstream QA correctness, and GLM outperformed Qwen on matched QA quality. Missing executions remain unavailable rather than being scored as zero.

GEMINI-2 SEMANTIC TOP-3

94.4%

USABLE QA OUTPUTS

71/72

GLM-PERSIAN READABILITY

80.0%

HUMAN-SCORED OUTPUTS

80/90

Controlled design

Three repositories, three embedding arms, and two LLM arms were evaluated with frozen questions and fixed generation settings.

Measured finding

Gemini 2 produced the strongest semantic retrieval, while GLM produced stronger QA quality and Persian readability than the local Qwen arm.

Availability rule

Unavailable executions remain unavailable. They are never converted into zero-valued quality scores.

View 48 benchmark questions

12 QA questions include evaluated answers

Human-reviewed, fixed-dataset measurements. Missing executions are reported separately from measured quality.

شکل ۴-۲: داشبورد نتایج نهایی ارزیابی پایان‌نامه

۴-۵- نتایج نمایه‌سازی و قابلیت اعتماد داده‌ها

در هر نه ترکیب حاصل از سه مخزن و سه مدل تعبیه‌سازی، تعداد بردارهای ذخیره‌شده با تعداد قطعه‌های قطعی SQLite برابر بود. این تطابق نشان می‌دهد همه قطعه‌های معتبر دارای بردار متناظر بودند و هیچ بردار بدون رکورد قطعی در نمایه فعال باقی نماند. نتیجه اعتبارسنجی نمایه‌ها در جدول ۳-۴ ارائه شده است.

جدول ۴-۳: نتیجه اعتبارسنجی نمایه‌ها

نتیجه	بردار در هر بازوی تعبیه‌سازی	قطعه‌های قطعی	مخزن
کامل	۴۰۹	۴۰۹	Hospital-System
کامل	۱۴۴	۱۴۴	CS-Bookstore
کامل	۱۲۳۶	۱۲۳۶	CodeCompass

شناسه قطعه، مسیر فایل، نام نماد و محدوده خطوط از SQLite دریافت شدند و مدل زبانی در ایجاد آن‌ها دخالت نداشت. سازوکار نمایه نامزد نیز از فعال شدن نمایه ناقص جلوگیری کرد و در شکست کنترل شده، نمایه معتبر قبلی را حفظ نمود. در نتیجه، ضعف احتمالی متن مدل نمی‌تواند هویت منبع یا محدوده رسمی ارجاع را تغییر دهد.

۴-۶- نتایج ارزیابی بازیابی

۴-۶-۱- نتایج کلی

جدول ۴-۴: نتایج کلی بازیابی روی ۳۶ پرسش

مدل	روش	Hit@1	Hit@3	Hit@5	Hit@10	MRR@10
Nomic محلی	واژگانی	%۱۱,۱	%۳۳,۳	%۵۰,۰	%۶۱,۱	۰,۲۴۲
Nomic محلی	معنایی	%۲۷,۸	%۳۶,۱	%۳۸,۹	%۴۷,۲	۰,۳۲۸
Nomic محلی	ترکیبی	%۳۳,۳	%۴۴,۴	%۴۷,۲	%۵۸,۳	۰,۴۰۶
Gemini 001	واژگانی	%۱۱,۱	%۳۳,۳	%۵۰,۰	%۶۱,۱	۰,۲۴۲
Gemini 001	معنایی	%۶۳,۹	%۸۶,۱	%۹۴,۴	%۱۰۰	۰,۷۷۱
Gemini 001	ترکیبی	%۵۵,۶	%۷۷,۸	%۸۸,۹	%۹۴,۴	۰,۶۸۹
Gemini 2	واژگانی	%۱۱,۱	%۳۳,۳	%۵۰,۰	%۶۱,۱	۰,۲۴۲
Gemini 2	معنایی	%۷۵,۰	%۹۴,۴	%۹۷,۲	%۱۰۰	۰,۸۵۳
Gemini 2	ترکیبی	%۴۷,۲	%۸۳,۳	%۹۴,۴	%۹۷,۲	۰,۶۶۱

بهترین نتیجه معنایی متعلق به Gemini Embedding2 بود که Hit@1 برابر ۷۵ درصد، Hit@3 برابر

۹۴٫۴ درصد و MRR@10 برابر ۰٫۸۵۳ ثبت کرد. باین حال، Gemini Embedding001 در روش ترکیبی Hit@1 و MRR@10 بالاتری داشت؛ بنابراین نتایج یک مبادله میان مدل تعبیه سازی و روش ادغام رتبه ها را نشان می دهند، نه برتری مطلق یک مدل در تمام معیارها. یکسان ماندن نتایج واژگانی در هر سه بازو نیز کنترل آزمایش را تأیید می کند.

۴-۶-۲- نتایج دوزبانه

جدول ۴-۵: عملکرد بازیابی بر اساس زبان پرسش

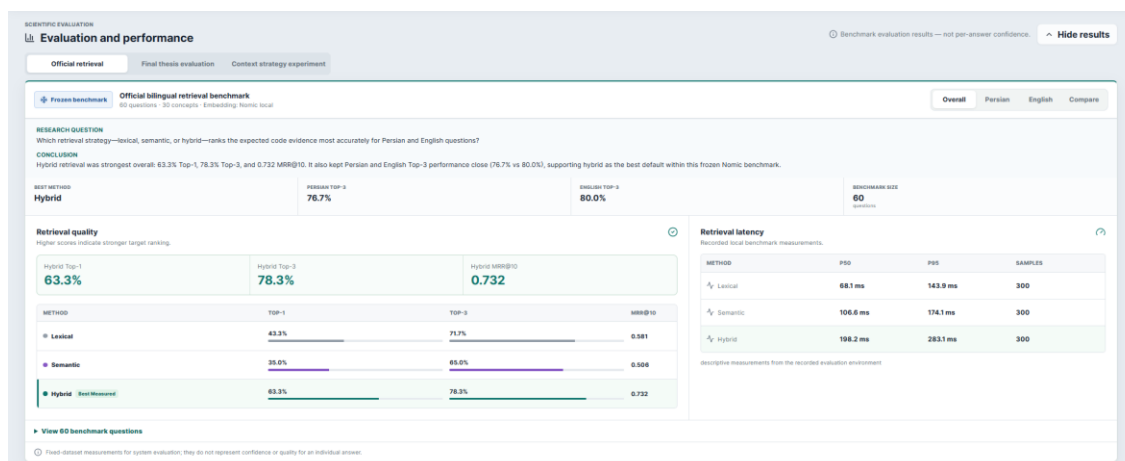
زبان	مدل	Hit@3 معنایی	MRR معنایی	Hit@3 ترکیبی	MRR ترکیبی
انگلیسی	Nomic	٪۷۲٫۲	۰٫۶۴۷	٪۸۳٫۳	۰٫۷۴۱
انگلیسی	Gemini 001	٪۸۸٫۹	۰٫۸۰۳	٪۸۸٫۹	۰٫۸۱۹
انگلیسی	Gemini 2	٪۹۴٫۴	۰٫۸۶۸	٪۸۸٫۹	۰٫۷۴۳
فارسی	Nomic	٪۰٫۰	۰٫۰۰۹	٪۵٫۶	۰٫۰۷۱
فارسی	Gemini 001	٪۸۳٫۳	۰٫۷۳۹	٪۶۶٫۷	۰٫۵۵۹
فارسی	Gemini 2	٪۹۴٫۴	۰٫۸۳۸	٪۷۷٫۸	۰٫۵۷۹

ضعف Nomic در اتصال پرسش فارسی به شناسه های انگلیسی آشکار بود: Hit@3 معنایی آن برای پرسش های فارسی صفر بود، در حالی که این مقدار برای Gemini Embedding001 و Gemini Embedding2 به ترتیب ۸۳٫۳ و ۹۴٫۴ درصد شد. این نتیجه نشان می دهد توانایی چندزبانه مدل تعبیه سازی یکی از عوامل اصلی موفقیت CodeCompass در بازیابی دوزبانه است.

۴-۶-۳- زمان بازیابی

در محیط ثبت شده، میانه زمان روش معنایی بین ۱۱٫۳ تا ۱۲٫۶ میلی ثانیه و میانه روش ترکیبی بین ۳۶٫۳ تا ۴۳٫۲ میلی ثانیه بود. روش ترکیبی به دلیل اجرای دو مسیر بازیابی و ادغام رتبه ها زمان بیشتری مصرف کرد. این اعداد توصیف محیط آزمایش هستند و تضمین عملکرد روی سخت افزار یا ارائه دهنده دیگر محسوب نمی شوند.

نمای کلی معیارهای کیفیت و تأخیر بازیابی در داشبورد ارزیابی رسمی در شکل ۴-۳ نمایش داده شده است.



شکل ۴-۳: داشبورد نتایج ارزیابی رسمی بازیابی

۴-۶-۴- مجموعه ارزیابی رسمی بازیابی با ۶۰ پرسش

مجموعه ارزیابی رسمی بازیابی CodeCompass شامل ۶۰ پرسش فارسی و انگلیسی متناظر با ۳۰ مفهوم روی سه مخزن ثابت شده Flask، itsdangerous و MarkupSafe بود. برای هر پرسش، روش‌های واژگانی، معنایی و ترکیبی اجرا شدند و در مجموع ۱۸۰ رکورد پایه تولید شد. در مقایسه کنترل شده مدل‌های تعبیه‌سازی، نسخه مخزن‌ها، پرسش‌ها، قطعه‌ها، شناسه قطعه، بازیابی واژگانی، عمق بازیابی و تنظیمات ادغام RRF ثابت ماندند. این مجموعه ارزیابی از ارزیابی نهایی ۳۶ پرسشی فصل حاضر مستقل است.

در بازوی محلی Nomic، روش واژگانی Hit@1 برابر ۴۳٫۳ درصد، Hit@3 برابر ۷۱٫۷ درصد و MRR@10 برابر ۰٫۵۸۱؛ روش معنایی به ترتیب ۳۵، ۶۵ و ۰٫۵۰۶؛ و روش ترکیبی به ترتیب ۶۳٫۳، ۷۸٫۳ و ۰٫۷۳۲ ثبت کرد. با جایگزینی کنترل شده مدل تعبیه‌سازی با Embedding001 Gemini، Hit@3 و MRR@10 معنایی به ۹۳٫۳ درصد و ۰٫۸۲۶ و مقادیر ترکیبی به ۹۵ درصد و ۰٫۸۳۱ رسیدند، درحالی‌که نتایج واژگانی بدون تغییر باقی ماندند. این آزمایش فقط کیفیت بازیابی را اندازه‌گیری کرد و نتیجه آن به‌تنهایی درباره کیفیت پاسخ نهایی داوری نمی‌کند.

۴-۷- نتایج پرسش و پاسخ مستند

۴-۷-۱- قابلیت اجرای مدل‌ها

در مجموع ۷۲ ترکیب پرسش، مدل تعبیه‌سازی و مدل زبانی اجرا شد. ۵۱ پاسخ در اجرای نخست موفق بودند و ۲۰ پاسخ دیگر با تلاش‌های بازیابی شده جداگانه تکمیل شدند. در نهایت ۷۱ پاسخ، معادل ۹۸٫۶ درصد، قابل استفاده بود. تنها خروجی نهایی ناموجود به پاسخ خالی ارائه‌دهنده مربوط می‌شد. هفت پاسخ قابل استفاده نیز با علامت محدودیت طول پایان یافتند و بدون پنهان کردن این وضعیت در مجموعه باقی ماندند. قابلیت اجرای مدل‌ها در پرسش و پاسخ در جدول ۴-۶ ارائه شده است.

جدول ۴-۶: قابلیت اجرای مدل‌ها در پرسش و پاسخ

مدل	موفقیت اولیه	بازیابی بعدی	شکست نهایی	کل
Qwen محلی	۲۷	۹	۰	۳۶
GLM 5.3 Flash	۲۴	۱۱	۱	۳۶
مجموع	۵۱	۲۰	۱	۷۲

۴-۷-۲- کیفیت پاسخ بر اساس مدل زبانی

جدول ۴-۷: میانگین ارزیابی انسانی کیفیت پاسخ‌ها

مدل	تعداد	صحت	اتکاپذیری	خوانایی فارسی	مفیدبودن
GLM 5.3 Flash	۳۵	۷,۹۴۳	۸,۳۱۴	۸,۰۰۰	۸,۲۸۶
Qwen محلی	۳۶	۶,۹۷۲	۷,۰۰۰	۵,۳۳۳	۶,۲۵۰

در ۳۵ جفت پاسخ با پرسش و شواهد یکسان، GLM نسبت به Qwen به‌طور میانگین یک نمره در صحت، ۱,۳۴۳ نمره در اتکاپذیری و ۲,۰۸۶ نمره در مفیدبودن بهتر بود. در ۱۷ جفت فارسی نیز برتری خوانایی GLM برابر ۲,۸۲۴ نمره بود. این نتیجه اثر مستقل توانایی مدل زبانی بر کیفیت پاسخ را نشان می‌دهد، هرچند مدل محلی برای کاربردهای آفلاین و حساس به حریم خصوصی همچنان ارزش دارد.

۴-۷-۳- ارتباط کیفیت بازیابی و پاسخ

بالاتر بودن معیارهای بازیابی الزاماً بهترین پاسخ نهایی را ایجاد نکرد. Gemini Embedding2 بهترین بازیابی معنایی را داشت، اما Gemini Embedding001 در این نمونه بالاترین میانگین صحت پاسخ را ثبت کرد. کیفیت پاسخ حاصل تعامل رتبه‌بندی شواهد، تنوع قطعه‌های انتخاب‌شده، محدودیت زمینه و توانایی مدل زبانی است؛ بنابراین ارزیابی بازیابی و تولید باید جداگانه تفسیر شوند.

۴-۷-۴- تحلیل خطا و توهم

در ارزیابی نهایی، ۷۳ خروجی بدون توهم، یک خروجی بدون توهم ولی مبهم، چهار خروجی دارای توهم صریح و دو خروجی دارای توهم خفیف یا ضمنی ثبت شدند. ضعف‌های اصلی شامل چند ادعای نادرست در خروجی‌های انگلیسی Qwen، خوانایی پایین‌تر فارسی Qwen، پایان برخی پاسخ‌ها به دلیل سقف طول و یک خروجی خالی GLM بود. رفتار شواهد ناکافی در تعدادی از موارد پوشش پاسخ را کاهش داد، اما از تولید ادعای

حدسی جلوگیری کرد.

۸-۴- نتایج مستندسازی تابع

در مستندسازی، نام و هویت تابع، مسیر، خطوط، پارامترها، نوع بازگشتی، استثنای صریح و فراخوانی‌های مستقیم به صورت قطعی استخراج شدند و مدل فقط توضیح طبیعی را تولید کرد. در ارزیابی نهایی چندمخزنی، GLM هر ۹ مورد را تکمیل کرد و میانگین صحت ۶۰۷۷۸، اتکاپذیری ۸۰۲۲۲، خوانایی فارسی ۸۰۱۱۱ و مفیدبودن ۸۰۱۱۱ به دست آمد. عدم تطابق هویت ارجاع صفر بود.

تمام ۹ اجرای Qwen در این مرحله با خطای ارائه‌دهنده ناموفق بودند؛ از این رو کیفیت آن ناموجود گزارش شد و صفر در نظر گرفته نشد. در ارزیابی مستقل ده‌موردی مستندسازی GLM، همه مسیرها، خطوط، پارامترها، انواع بازگشتی، استنهاها و مجموعه فراخوانی‌های مستقیم با منبع قطعی مطابقت داشتند و هیچ شناسه ساختاری جعلی دیده نشد. میانگین انسانی این مجموعه ۸۰۹ برای خوانایی فارسی، ۹۰۲ برای صحت و ۸۰۹ برای مفیدبودن بود.

این نتایج نشان می‌دهند جداسازی حقایق قطعی از نثر مدل، مرز اعتماد مؤثری ایجاد کرده است. تغییر کیفیت متن مدل نمی‌تواند هویت منبع یا حقایق ساختاری تأییدشده را تغییر دهد.

۹-۴- آزمایش مقایسه راهبردهای ساخت زمینه

۹-۴-۱- روش آزمایش

برای سنجش ضرورت معماری RAG، پنج راهبرد ارسال کل مخزن، RAG واژگانی، RAG معنایی با Gemini Embedding2، RAG ترکیبی و عامل OpenCode مقایسه شدند. آزمایش روی ۱۸ مفهوم و ۳۶ پرسش فارسی و انگلیسی در سه نسخه ثابت‌شده مخزن انجام شد و مدل مولد همه بازوها Flash 5.3 GLM بود.

عامل OpenCode کل مخزن را یکجا دریافت نمی‌کرد. برای هر پرسش یک نشست تازه داشت و با ابزارهای read، grep و glob فایل‌های لازم را به صورت پویا انتخاب می‌کرد. خروجی‌های ناموفق و تلاش‌های بازیابی‌شده رکوردهای جداگانه بودند و هیچ نتیجه خامی بازنویسی نشد.

۹-۴-۲- اعتبارسنجی انسانی کور مستقل

پس از ارزیابی اولیه خروجی‌ها با داور مدلی کور، یک ارزیاب انسانی مستقل ۵۴ پاسخ را بدون مشاهده نام راهبردها بررسی کرد. نمونه پیش از مشاهده نمره‌ها انتخاب شد و برای هر یک از سه مقایسه اصلی، ۱۲ جفت متوازن روی پرسش یکسان داشت. هیچ امتیازی مفقود نبود. پوشش حقایق، صحت، اتکاپذیری و کامل‌بودن در مقیاس صفر تا ده سنجیده شدند و کیفیت ترکیبی از میانگین این چهار معیار به دست آمد. فاصله اطمینان با

۲۰۰۰۰ باز نمونه گیری زوجی و مقدار احتمال با آزمون دقیق جابه جایی علامت دوطرفه محاسبه شد. نتایج اعتبارسنجی انسانی راهبردهای زمینه در جدول ۸-۴ ارائه شده است.

جدول ۴-۸: نتایج اعتبارسنجی انسانی راهبردهای زمینه

مقدار احتمال	فاصله اطمینان ۹۵٪	اختلاف	کیفیت مقابل	کیفیت معنایی	مقایسه
۰,۰۳۱	۲,۹۷۹- تا ۰,۱۸۷-	-۱,۳۷۵	۹,۹۵۸	۸,۵۸۳	معنایی در برابر کل مخزن
۰,۰۰۴	۶,۱۰۴+ تا ۲,۳۹۶+	۴,۲۵۰+	۴,۵۲۱	۸,۷۷۱	معنایی در برابر واژگانی
۰,۳۰۹	۲,۹۳۸+ تا ۰,۸۷۵-	۰,۹۳۸+	۷,۸۳۳	۸,۷۷۱	معنایی در برابر عامل

ارزیابی انسانی تأیید کرد که کل مخزن در پاسخ های موفق مخازن کوچک کیفیت خام بالاتری داشته است، ارزیابی معنایی به طور معنادار از ارزیابی واژگانی بهتر است و برای عامل OpenCode برتری معناداری نسبت به RAG معنایی مشاهده نمی شود.

۴-۹-۳- کل مخزن در برابر RAG معنایی

در نمونه انسانی ۱۲ جفتی، کیفیت ترکیبی کل مخزن ۹,۹۵۸ و کیفیت RAG معنایی ۸,۵۸۳ بود. اختلاف ۱,۳۷۵ نمره ای به نفع کل مخزن، فاصله اطمینان ۹۵ درصد از ۰,۱۸۷- تا ۲,۹۷۹+ نمره و مقدار احتمال برابر ۰,۰۳۱ داشت. کل مخزن هر ۱۲ پاسخ را کامل تولید کرد؛ RAG معنایی ده پاسخ کامل، یک پاسخ جزئی و یک پاسخ نادرست داشت.

این برتری فقط به پاسخ های موفق مخازن کوچک مربوط است. ارسال کامل CodeCompass با ۱۰۰ فایل و حدود ۱۸۰ هزار توکن توسط درگاه به طور پایدار پذیرفته نشد و هر ۱۲ اجرای آن ناموجود ماند. RAG معنایی در کل ۳۳ پاسخ از ۳۶ پرسش تولید کرد و ورودی متوسط پاسخ موفق را از حدود ۱۷۴۴۷ به ۸۲۷ توکن، یعنی نزدیک به ۹۵ درصد، کاهش داد. بنابراین کل مخزن در مقیاس کوچک دقیق تر بود، اما RAG از نظر هزینه، زمان و قابلیت اجرا روی مخزن بزرگ راهکار عملی تری است.

۴-۹-۴- سودمندی تعبیه سازی

بازخوانی شواهد در روش واژگانی ۲۲,۲۲ درصد و در روش معنایی ۶۴,۷۲ درصد بود. اختلاف ۴۲,۵ واحد درصدی دارای فاصله اطمینان ۹۵ درصد از ۲۸,۳۳ تا ۵۶,۹۴ واحد درصد بود. در ارزیابی ماشینی ۳۰ پاسخ زوجی نیز روش معنایی پوشش حقایق را ۴,۲۸۱، صحت را ۳,۸۱۳ و اتکاپذیری را ۰,۴۶۹ نمره افزایش داد. داور انسانی مستقل این نتیجه را در نمونه ۱۲ جفتی تأیید کرد. کیفیت ترکیبی معنایی ۸,۷۷۱ و واژگانی

۴,۵۲۱ بود؛ اختلاف ۴,۲۵ نمره، فاصله اطمینان ۹۵ درصد از ۲,۳۹۶ تا ۶,۱۰۴ و مقدار احتمال برابر ۰,۰۰۰۴ داشت. روش معنایی در ۹ جفت بهتر و در سه جفت مساوی بود و هیچ باختی نداشت. اختلاف انسانی پوشش حقایق، صحت، اتکاپذیری و کامل بودن به ترتیب ۵, ۳,۲۵, ۳,۷۵ و ۵ نمره به نفع روش معنایی بود. هزینه ثبت شده واژگانی ۰,۰۰۶۴۲۲ دلار و معنایی ۰,۰۰۶۸۶۷ دلار بود و میانه زمان از ۱۴,۲۰۹ به ۱۵,۹۹۹ ثانیه رسید. این افزایش محدود در برابر بهبود بازیابی و کیفیت پاسخ نشان می دهد تعبیه سازی یک مؤلفه ضروری و قابل دفاع در CodeCompass است.

۴-۹-۵- عامل OpenCode در برابر RAG معنایی

در نمونه انسانی ۱۲ جفتی، کیفیت ترکیبی RAG معنایی ۸,۷۷۱ و عامل ۷,۸۳۳ بود. اختلاف ۰,۹۳۸ نمره ای به نفع RAG معنایی بود، اما فاصله اطمینان از ۰,۸۷۵- تا ۲,۹۳۸+ و مقدار احتمال برابر ۰,۳۰۹ بود؛ بنابراین شواهد کافی برای برتری معنادار هیچ یک از دو روش وجود ندارد. RAG معنایی ده پاسخ کامل و دو پاسخ جزئی داشت، در حالی که عامل نه پاسخ کامل و سه پاسخ نادرست تولید کرد. عامل در اجرای نخست ۳۳ پاسخ از ۳۶ پرسش تولید کرد و سه شکست آن در تلاش های جداگانه بازیابی شدند. همچنین هر ۱۲ پرسش CodeCompass را پاسخ داد. باین حال، کیفیت آن میان مخازن یکنواخت نبود و در چند پرسش Hospital-System به جای پاسخ دقیق، نمایی عمومی از مخزن ارائه کرد. مزیت اصلی عامل، دسترسی پویای آن به مخزن بزرگ بود، نه کیفیت اثبات شده بالاتر.

۴-۹-۶- هزینه و زمان راهبردها

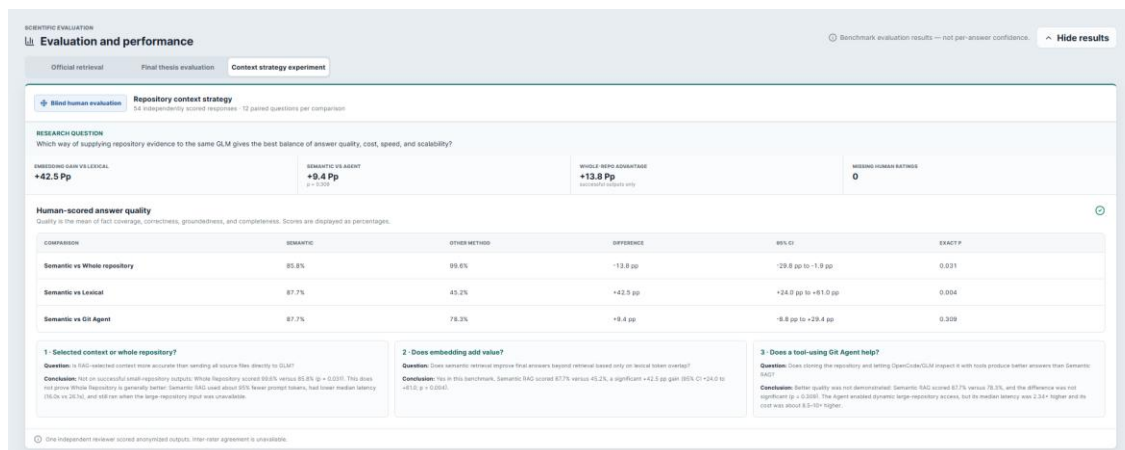
جدول ۴-۹: هزینه، زمان و قابلیت اجرای راهبردها

راهبرد	موفقیت مؤثر	میانه زمان	صدک ۹۵	هزینه
کل مخزن	۲۳ از ۳۶؛ ۱۲ ناموجود	۲۶,۰۸۳ ثانیه	۴۰,۵۴۳ ثانیه	۰,۰۲۹۱۴۲ دلار
RAG واژگانی	۳۲ از ۳۶	۱۴,۲۰۹ ثانیه	۲۷,۰۶۶ ثانیه	۰,۰۰۶۴۲۲ دلار
RAG معنایی	۳۳ از ۳۶	۱۵,۹۹۹ ثانیه	۳۰,۸۸۴ ثانیه	۰,۰۰۶۸۶۷ دلار
عامل OpenCode	۳۶ از ۳۶ پس از بازیابی	۳۷,۴۰۹ ثانیه	۶۲,۰۷۶ ثانیه	۰,۰۰۶۸۴۸۳ دلار

عامل حدود ۲,۳۴ برابر کندتر از RAG معنایی بود. هزینه کل ثبت شده آن ۹,۹۷ برابر و هزینه تقریبی هر پاسخ موفق ۸,۵۳ برابر بیشتر بود. عامل برای پاسخ های مؤثر ۲۱۶ بار فایل خواند، ۶۶ جست و جوی متنی و ۵۰ عملیات یافتن مسیر انجام داد و حدود ۱,۵۳ میلیون توکن رویداد مصرف کرد. بنابراین عامل ابزارمحور می تواند مکمل پرسش های بسیار گسترده یا شکست بازیابی باشد، اما انتخاب پیش فرض اقتصادی تری از RAG

معنایی نیست.

جمع‌بندی بصری مقایسه کیفیت پاسخ، فاصله اطمینان و مقدار احتمال راهبردهای ساخت زمینه در شکل ۴-۴ آمده است.



شکل ۴-۴: داشبورد مقایسه راهبردهای ساخت زمینه

۴-۱۰- بحث و تفسیر نتایج

نتایج نشان دادند توانایی چندزبانه مدل تعبیه‌سازی یکی از گلوگاه‌های اصلی بازیابی است. مدل محلی Nomic در انگلیسی قابل استفاده بود، اما در اتصال پرسش فارسی به کد انگلیسی ضعف جدی داشت. مدل‌های Gemini این فاصله زبانی را کاهش دادند. باین‌حال، بهترین بازیاب الزاماً بهترین پاسخ نهایی را تولید نکرد؛ زیرا کیفیت پاسخ به رتبه شواهد، محدودیت زمینه و توان مدل زبانی نیز وابسته است.

ارسال کل مخزن در مخازن کوچک بیشترین کیفیت خام را ایجاد کرد، ولی با رشد مخزن از نظر اندازه ورودی و قابلیت پذیرش درگاه پایدار نبود. RAG با انتخاب بخش کوچکی از منبع، هزینه و زمان را کاهش داد و امکان پاسخ‌گویی روی CodeCompass را حفظ کرد. مقایسه انسانی معنایی و واژگانی نیز نشان داد سود تعبیه‌سازی مستقل از داور مدلی قابل مشاهده است.

عامل OpenCode توانست مخزن بزرگ را به‌صورت پویا بررسی کند، اما برتری کیفی آن نسبت به RAG معنایی اثبات نشد و هزینه عملیاتی بسیار بیشتری داشت. بر اساس این شواهد، RAG معنایی مناسب‌ترین مسیر پیش‌فرض CodeCompass است؛ بازیابی واژگانی گزینه‌ای بدون نیاز به تعبیه‌سازی، کل مخزن مرجعی برای مخازن کوچک و عامل ابزارمحور گزینه‌ای مکمل برای شرایط خاص باقی می‌ماند.

۴-۱۱- محدودیت‌ها و تهدیدهای اعتبار

ارزیابی نهایی ۳۶ پرسشی و آزمایش راهبرد زمینه به سه مخزن پایتون، ۱۸ مفهوم دوزبانه، ۱۲ پرسش پایه تولید پاسخ و نسخه‌ها و تنظیمات ثبت‌شده مدل‌ها محدود هستند. مجموعه ارزیابی رسمی ۶۰ پرسشی نیز

سه مخزن دیگر و ۳۰ مفهوم دوزبانه را پوشش می‌دهد و فقط کیفیت بازیابی را می‌سنجد؛ بنابراین هیچ‌یک مبنای تعمیم به همه مخازن یا زبان‌ها نیستند. ترجمه فارسی و انگلیسی یک مفهوم دو مشاهده کاملاً مستقل محسوب نمی‌شود. زمان و هزینه نیز به سخت‌افزار، شبکه، ارائه‌دهنده و قیمت زمان اجرای آزمایش وابسته‌اند. ارزیابی انسانی اصلی پایان‌نامه و اعتبارسنجی آزمایش راهبرد زمینه هرکدام یک داور مستقل داشتند؛ بنابراین توافق میان چند داور اندازه‌گیری نشده است. اعتبارسنجی اخیر ۵۴ پاسخ و ۱۲ جفت برای هر مقایسه را پوشش داد، نه همه خروجی‌ها. نتیجه کل مخزن نیز مشروط به پاسخ‌های موفق است و ۱۲ اجرای ناموجود CodeCompass را در میانگین کیفیت پنهان نمی‌کند.

یک پاسخ GLM و همه نتایج Qwen در ارزیابی نهایی مستندسازی ناموجود بودند و برای آن‌ها کیفیت تخمین زده نشد. آزمون پایداری چندباره آزمایش راهبرد زمینه نیز به دلیل خطاهای ارائه‌دهنده کامل نشد؛ از این رو تکرارپذیری کامل متن پاسخ ادعا نمی‌شود. عامل OpenCode خود نوعی بازیابی پویا است و مقایسه آن با RAG آزمایش خالص اثر تعبیه‌سازی نیست.

۴-۱۲- جمع‌بندی

در این فصل، نسخه نهایی CodeCompass در لایه‌های نمایه‌سازی، بازیابی، پرسش‌وپاسخ و مستندسازی ارزیابی شد. هر نه نمایه‌برداری با داده‌های قطعی SQLite تطابق کامل داشتند و ارجاع‌های مسیر، نماد و خطوط مستقل از متن مدل باقی ماندند. Gemini Embedding2 بهترین بازیابی معنایی را با $\text{Hit}@1$ برابر ۷۵ درصد، $\text{Hit}@3$ برابر ۹۴٫۴ درصد و $\text{MRR}@10$ برابر ۰٫۸۵۳ ثبت کرد و مدل GLM در مقایسه زوجی پاسخ، کیفیت بالاتری از Qwen داشت. در مجموعه ارزیابی رسمی ۶۰ پرسشی نیز روش ترکیبی محلی $\text{Hit}@3$ برابر ۷۸٫۳ درصد و $\text{MRR}@10$ برابر ۰٫۷۳۲ داشت و جایگزینی کنترل‌شده تعبیه‌سازی با Gemini Embedding001 این مقادیر را به ۹۵ درصد و ۰٫۸۳۱ رساند.

آزمایش راهبرد زمینه نشان داد کل مخزن در پاسخ‌های موفق مخازن کوچک کیفیت خام بالاتری دارد، ولی برای CodeCompass بزرگ پایدار نبود. RAG معنایی با نزدیک به ۹۵ درصد ورودی کمتر، هزینه و زمان پایین‌تر و قابلیت استفاده در مخزن بزرگ، راهکار عملی‌تری بود. ارزیابی انسانی کور نیز کیفیت معنایی را ۴٫۲۵ نمره بالاتر از واژگانی ارزیابی کرد و سود تعبیه‌سازی را با مقدار احتمال برابر ۰٫۰۰۴ تأیید نمود.

عامل OpenCode به مخزن بزرگ دسترسی موفق داشت، اما برتری معناداری نسبت به RAG معنایی نشان نداد و حدود ۲٫۳۴ برابر کندتر و ۸٫۵ تا ۱۰ برابر پرهزینه‌تر بود. در نتیجه، شواهد از انتخاب RAG معنایی به‌عنوان مسیر پیش‌فرض سامانه حمایت می‌کنند. CodeCompass توانسته است پرسش‌های فارسی و انگلیسی را به شواهد ساختاریافته کد متصل کند، پاسخ و مستندات مبتنی بر شواهد تولید نماید و امکان بررسی مستقیم منبع را در اختیار کاربر قرار دهد.

فصل پنجم

نتیجه‌گیری و پیشنهادها

۵-۱- نتیجه‌گیری کلی

این پروژه با هدف طراحی و پیاده‌سازی سامانه‌ای محلی‌محور برای درک مخازن کد پایتون شکل گرفت؛ سامانه‌ای که بتواند پرسش فارسی یا انگلیسی کاربر را به شواهد واقعی موجود در کد متصل کند و پاسخ را همراه با مسیر فایل، نام نماد و محدوده خطوط قابل بررسی ارائه دهد. مسئله اصلی فقط تولید توضیح طبیعی درباره کد نبود، بلکه ایجاد مرزی روشن میان اطلاعات قطعی استخراج‌شده از مخزن و متن احتمالی مدل زبانی بود. بر همین اساس، CodeCompass از تحلیل نحوی، قطعه‌بندی ساختارآگاه، ذخیره‌سازی فراداده قطعی، بازیابی چندروشی و تولید پاسخ مبتنی بر شواهد استفاده می‌کند.

نتایج پیاده‌سازی و ارزیابی نشان دادند که این معماری می‌تواند جریان کامل انتخاب مخزن، پویش امن، تحلیل ساختاری، نمایه‌سازی، جست‌وجو، پرسش‌وپاسخ، مستندسازی تابع و مشاهده کد مورد استناد را در یک محیط یکپارچه فراهم کند. در این جریان، SQLite مرجع اصلی هویت فایل، نماد، قطعه و ارجاع است و نمایه برداری فقط برای یافتن شواهد به کار می‌رود. در نتیجه، حتی اگر متن تولیدی مدل ناقص باشد، هویت منبع و خطوط ارجاع از خروجی مدل ساخته نمی‌شوند و قابلیت راستی‌آزمایی پاسخ حفظ می‌شود.

ارزیابی نهایی همچنین نشان داد که انتخاب مدل و راهبرد بازیابی باید بر مبنای زبان پرسش، اندازه مخزن و محدودیت منابع انجام شود. مدل‌های تعبیه‌سازی چندزبانه در اتصال پرسش فارسی به شناسه‌ها و کد انگلیسی عملکرد بهتری داشتند و بازیابی معنایی در مقایسه با بازیابی صرفاً واژگانی، شواهد مرتبط‌تر و پاسخ‌های باکیفیت‌تری تولید کرد. ارسال کل مخزن برای مخازن کوچک می‌تواند کیفیت خام بالاتری داشته باشد، اما با بزرگ‌شدن مخزن از نظر ظرفیت ورودی، هزینه و پایداری محدود می‌شود. بنابراین، بازیابی معنایی مبتنی بر زمینه منتخب، انتخاب پیش‌فرض متعادل‌تری برای سامانه نهایی است.

۵-۲- میزان تحقق اهداف پروژه

هدف نخست پروژه، استخراج ساختار قابل اعتماد از مخازن پایتون بود. این هدف با محدودکردن پویش

به ریشه انتخاب شده، حذف مسیرها و فایل های حساس یا نامرتبط، تحلیل درخت نحو انتزاعی و استخراج ماژول ها، کلاس ها، توابع، متدها، امضاها، پارامترها، توضیحات مستند و محدوده خطوط تحقق یافت. قطعه های حاصل به شناسه و هش پایدار مجهز شدند تا ارتباط میان متن قطعه، فایل و نماد در نمایه سازی اولیه و افزایشی حفظ شود.

هدف دوم، ایجاد لایه ذخیره سازی و نمایه سازی بازیابی پذیر بود. نگهداری فراداده قطعی در SQLite و استفاده از ChromaDB به عنوان نمایه برداری قابل جایگزینی، وابستگی اطلاعات هویتی به موتور برداری را از بین برد. ساخت نمایه نامزد، کنترل هویت ارائه دهنده و مدل تعبیه سازی، اعتبارسنجی تعداد و ابعاد بردارها و فعال سازی فقط پس از موفقیت کامل باعث شد شکست یک نمایه جدید، نمایه معتبر قبلی را از دسترس خارج نکند. تطابق کامل تعداد قطعه ها و بردارها در هر نه ترکیب ارزیابی شده، صحت این طراحی را تأیید کرد.

هدف سوم، فراهم کردن بازیابی و پاسخ گویی مستند بود. سه روش واژگانی، معنایی و ترکیبی پیاده سازی شدند و کاربر می تواند متناسب با نوع پرسش از آن ها استفاده کند. پاسخ مدل تنها از زمینه محدود حاصل از بازیابی تولید می شود و در نبود شواهد کافی، سامانه به جای حدس زدن وضعیت شواهد ناکافی را اعلام می کند. ارجاع ها نیز پس از تولید پاسخ و بر اساس رکوردهای قطعی ساخته می شوند. این سازوکار همراه با نمایش مستقیم خطوط در ویرایشگر کد، امکان بررسی ادعاهای پاسخ را برای کاربر فراهم می سازد.

هدف چهارم، تولید مستندات تابع با جداسازی حقایق ساختاری از توضیح مدل بود. نام تابع، مسیر، محدوده خطوط، پارامترها، نوع بازگشتی، استثنای صریح و فراخوانی های مستقیم به صورت قطعی استخراج می شوند و مدل زبانی فقط بیان طبیعی این اطلاعات را تولید می کند. افزون بر این، استقلال ارائه دهنده تعبیه سازی از مدل زبانی، اجرای کاملاً محلی با Ollama یا استفاده کنترل شده از ارائه دهندگان سازگار با OpenAI را ممکن کرد. رابط FastAPI و رابط کاربری React و Vite نیز همه این قابلیت ها را در اختیار کاربر قرار دادند. بر این مبنا، اهداف کارکردی و غیرکارکردی اصلی پروژه در محدوده تعریف شده تحقق یافته اند.

۵-۳- پاسخ به پرسش های اصلی ارزیابی

۵-۳-۱- انتخاب زمینه با RAG در برابر ارسال کل مخزن

نتیجه این مقایسه مطلق نیست و به اندازه مخزن وابسته است. در پاسخ های موفق مخازن کوچک، ارسال کل مخزن امتیاز انسانی ۹۰۹۵۸ را در برابر ۸۰۵۸۳ برای RAG معنایی به دست آورد و اختلاف به نفع کل مخزن معنادار بود. با این حال، این راهبرد برای مخزن CodeCompass با حدود ۱۸۰ هزار توکن به طور پایدار قابل اجرا نبود و تمام اجراهای آن ناموجود ماند. RAG معنایی با نزدیک به ۹۵ درصد ورودی کمتر، زمان و هزینه پایین تر و امکان پاسخ گویی در مخزن بزرگ، مقیاس پذیری عملی بیشتری نشان داد. بنابراین، ارسال کل مخزن فقط برای مخازن کوچک و در صورت پذیرش کامل ورودی می تواند انتخاب مناسبی باشد؛ برای کاربرد

عمومی CodeCompass، انتخاب زمینه با RAG راهبرد پیش فرض حرفه‌ای‌تر و پایدارتر است.

۵-۳-۲- سودمندی تعبیه‌سازی در مقایسه با بازیابی واژگانی

تعبیه‌سازی در مجموعه آزمون این پروژه سود قابل اندازه‌گیری داشت. بازخوانی شواهد در بازیابی معنایی ۶۴٫۷۲ درصد و در بازیابی واژگانی ۲۲٫۲۲ درصد بود؛ یعنی روش معنایی ۴۲٫۵ واحد درصد بهبود ایجاد کرد. در ارزیابی انسانی زوجی نیز کیفیت ترکیبی پاسخ معنایی ۸۰۷۷۱ و پاسخ واژگانی ۴۰۵۲۱ بود. اختلاف ۴۰۲۵ نمره با مقدار احتمال ۰٫۰۰۴ معنادار بود و روش معنایی در ۹ جفت بهتر، در سه جفت مساوی و در هیچ جفتی ضعیف‌تر نبود. افزایش محدود زمان و هزینه در برابر این بهبود نشان می‌دهد نمایه برداری و بازیابی معنایی، به‌ویژه برای پرسش فارسی درباره کد دارای شناسه‌های انگلیسی، مؤلفه‌ای ضروری در سامانه است؛ هرچند بازیابی واژگانی برای نام‌های دقیق همچنان مفید باقی می‌ماند.

۵-۳-۳- عامل ابزارمحور در برابر RAG معنایی

عامل OpenCode با ابزارهای خواندن فایل و جست‌وجو توانست مخزن بزرگ را به‌صورت پویا بررسی کند، اما برتری کیفی آن نسبت به RAG معنایی اثبات نشد. امتیاز انسانی RAG معنایی ۸۰۷۷۱ و امتیاز عامل ۷۰۸۳۳ بود و مقدار احتمال ۰٫۳۰۹ شواهد کافی برای تفاوت معنادار فراهم نکرد. در مقابل، عامل حدود ۲۰۳۴ برابر کندتر و هزینه آن بسته به مبنای مقایسه حدود ۸۰۵ تا ۱۰ برابر بیشتر بود. بنابراین، استفاده از عامل ابزارمحور به‌عنوان مسیر پیش فرض توجیه نمی‌شود. این رویکرد می‌تواند در آینده برای پرسش‌های گسترده یا چندمرحله‌ای که زمینه منتخب معمولی برای آن‌ها کافی نیست، به‌صورت کنترل‌شده و اختیاری بررسی شود.

۵-۴- نوآوری و سهم پروژه

سهم اصلی این پروژه ارائه یک الگوریتم منفرد یا ادعای ابداع مدل جدید نیست، بلکه طراحی و اعتبارسنجی یک زنجیره یکپارچه برای درک قابل اعتماد کد است. در این زنجیره، مدل برای بازیابی معنایی و تولید توضیح به کار می‌رود، اما هویت فایل، نماد، هاش محتوای، محدوده خطوط و ارجاع از فراداده قطعی استخراج می‌شوند. این جداسازی، جعل ارجاع توسط مدل را از مسیر عادی سامانه حذف می‌کند و کاربر می‌تواند هر پاسخ را مستقیماً با کد منبع تطبیق دهد.

سهم دوم، ترکیب تحلیل ساختاری پایتون با قطعه‌بندی ساختارآگاه، شناسه‌های پایدار و نمایه‌سازی افزایشی و امن است. این طراحی به سامانه اجازه می‌دهد تنها تغییرهای لازم را پردازش کند، سازگاری میان SQLite و نمایه برداری را پیش از فعال‌سازی کنترل نماید و در صورت شکست، نسخه معتبر قبلی را حفظ کند. سهم سوم، پشتیبانی دوزبانه فارسی و انگلیسی در یک سامانه محلی‌محور و مستقل از یک ارائه‌دهنده خاص است؛ به‌گونه‌ای که مدل تعبیه‌سازی و مدل زبانی می‌توانند جداگانه انتخاب شوند.

سهم چهارم، ارزیابی چندلایه و مبتنی بر داده منجمد است. کیفیت نمایه، بازیابی، پاسخ، مستندسازی،

قابلیت اعتماد ارجاع، هزینه، زمان و اجراهای ناموجود جداگانه گزارش شدند و آزمایش مقایسه راهبرد زمینه به سه پرسش عملی درباره کل مخزن، تعبیه‌سازی و عامل ابزارمحور پاسخ داد. ثبت جداگانه تلاش‌های بازیابی شده و تبدیل نکردن خروجی ناموجود به نمره صفر نیز از مخدوش شدن نتیجه جلوگیری کرد. بنابراین، نوآوری پروژه را باید در معماری اعتماد، یکپارچه‌سازی قابلیت‌ها و اثبات تجربی تصمیم‌های اصلی دانست، نه در ادعای برتری عمومی بر همه ابزارها و داده‌ها.

۵-۵- محدودیت‌ها

نسخه کنونی فقط مخازن پایتون را تحلیل می‌کند و استخراج ساختار آن به امکانات درخت نحو انتزاعی این زبان وابسته است. ارزیابی نهایی و آزمایش راهبرد زمینه به سه مخزن، ۱۸ مفهوم دوزبانه و تنظیمات و نسخه‌های مشخص مدل‌ها محدود بودند. مجموعه ارزیابی رسمی مستقل نیز سه مخزن و ۳۰ مفهوم دوزبانه دیگر را فقط در لایه بازیابی پوشش داد؛ از این رو تعمیم اعداد به همه مخازن یا ارائه‌دهندگان ممکن نیست. هر ارزیابی انسانی توسط یک داور مستقل انجام شد و توافق میان چند داور اندازه‌گیری نشد. همچنین، زمان و هزینه ثبت شده به سخت‌افزار، شبکه، محدودیت درگاه و قیمت ارائه‌دهنده در زمان آزمایش وابسته است. کیفیت بازیابی معنایی به توانایی مدل تعبیه‌سازی وابسته است و نتایج مدل محلی Nomic نشان داد که هر مدل تعبیه‌سازی برای اتصال پرسش فارسی به کد انگلیسی مناسب نیست. اجرای محلی از حریم خصوصی پشتیبانی می‌کند، اما به منابع سخت‌افزاری و در دسترس بودن مدل نیاز دارد؛ استفاده از ارائه‌دهنده بیرونی نیز مستلزم پذیرش ارسال قطعه‌های منتخب است. سامانه فعلی گراف کامل فراخوانی یا وابستگی، مسیریابی خودکار پرسش و عامل چندمرحله‌ای را پیاده‌سازی نمی‌کند و برای پرسش‌های بسیار کلی ممکن است زمینه منتخب همه روابط لازم را پوشش ندهد.

۵-۶- پیشنهادها برای ادامه کار

نخست، می‌توان دامنه ارزیابی را با مخازن بیشتر، اندازه‌های متنوع‌تر، پرسش‌های جدید و چند ارزیاب انسانی گسترش داد و توافق میان ارزیابان را اندازه‌گیری کرد. بررسی مدل‌های تعبیه‌سازی محلی چندزبانه نیز می‌تواند وابستگی به سرویس بیرونی را کاهش دهد. افزودن زبان‌های برنامه‌نویسی دیگر باید پس از تعریف تحلیلگر ساختاری و قواعد قطعه‌بندی ویژه هر زبان انجام شود تا مرز اعتماد فعلی حفظ گردد.

دوم، یک مسیریاب سبک برای تشخیص نوع پرسش می‌تواند میان بازیابی واژگانی، معنایی و ترکیبی انتخاب کند و برای مخازن بسیار کوچک، ارسال کامل منبع را فقط در صورت قرارگرفتن در بودجه ورودی پیشنهاد دهد. بهبود بازرتبه‌بندی، تنوع نتایج، قطعه‌بندی و تخصیص تطبیقی بودجه زمینه نیز می‌تواند شواهد دقیق‌تری فراهم کند. این تغییرها باید با مجموعه آزمون ثابت و بدون تغییر هم‌زمان پرسش، زمینه و تنظیمات تولید ارزیابی شوند.

سوم، برای پرسش‌های گسترده یا چندمرحله‌ای می‌توان مسیر اختیاری RAG عامل‌محور را بررسی کرد؛ اما فعال‌سازی آن باید به تشخیص نیاز، سقف تعداد ابزار، بودجه زمان و هزینه و ثبت کامل عملیات محدود باشد. نتایج این پژوهش از جایگزینی مسیر پیش‌فرض با عامل پشتیبانی نمی‌کنند. ایجاد گراف سبک وابستگی میان ماژول‌ها، نمادها و فراخوانی‌های مستقیم نیز می‌تواند برای پرسش‌های معماری مفید باشد، مشروط بر آنکه داده گراف به‌صورت قطعی از کد استخراج شود و مرجع ارجاع همچنان SQLite باقی بماند.

در نهایت، یکپارچه‌سازی با محیط‌های توسعه، بازخورد کاربر درباره مفید بودن شواهد و نمایش روشن‌تر عدم قطعیت می‌تواند تجربه استفاده را بهبود دهد. مسیر توسعه آینده باید اصل مرکزی CodeCompass را حفظ کند: مدل می‌تواند در یافتن و توضیح شواهد کمک کند، اما ادعاهای قابل استناد باید به داده قطعی و کد قابل مشاهده متصل بمانند.

- [1] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search,” arXiv preprint arXiv:1909.09436, 2019. Available: <https://arxiv.org/abs/1909.09436>
- [2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020. Available: <https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [3] Python Software Foundation, “ast — Abstract Syntax Trees,” Python 3 Documentation. Accessed: Sep. 9, 2026. Available: <https://docs.python.org/3/library/ast.html>
- [4] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020, doi: 10.18653/v1/2020.findings-emnlp.139.
- [5] D. Guo et al., “GraphCodeBERT: Pre-Training Code Representations with Data Flow,” in *9th International Conference on Learning Representations (ICLR)*, 2021. Available: <https://openreview.net/forum?id=jLoC4ez43PZ>
- [6] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, “UniXcoder: Unified Cross-Modal Pre-training for Code Representation,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pp. 7212–7225, 2022, doi: 10.18653/v1/2022.acl-long.499.
- [7] Chroma, “Collections: Distance Metric,” Chroma Cookbook. Accessed: Sep. 9, 2026. Available: <https://cookbook.chromadb.dev/core/collections/>
- [8] X. Li et al., “CodeRetriever: A Large Scale Contrastive Pre-Training Method for Code Search,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 2898–2910, 2022, doi: 10.18653/v1/2022.emnlp-main.187.
- [9] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759, 2009, doi: 10.1145/1571941.1572114.
- [10] F. Zhang et al., “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2471–2484, 2023, doi: 10.18653/v1/2023.emnlp-main.151.
- [11] V. Lomshakov, A. Podivilov, S. Savin, O. Baryshnikov, A. Lisevych, and S. Nikolenko, “ProConSuL: Project Context for Code Summarization with LLMs,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 866–880, 2024, doi: 10.18653/v1/2024.emnlp-industry.65.
- [12] R. Sekizawa, N. Duan, S. Lu, and H. Yanaka, “Constructing Multilingual Code Search Dataset Using Neural Machine Translation,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: Student Research Workshop*, pp. 69–75, 2023, doi: 10.18653/v1/2023.acl-srw.10.